



PES
UNIVERSITY

CELEBRATING 50 YEARS

BIG DATA

Hadoop Ecosystem and Apache SeaTunnel

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by Dr. K V Subramaniam's and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from Dr. H L Phalachandra's slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Apache Sqoop — A Legacy Tool

HADOOP ECOSYSTEM — RETIRED

Apache Sqoop (SQL-to-Hadoop) was a tool designed to transfer bulk data between Apache Hadoop (HDFS) and structured data stores such as relational databases (MySQL, Oracle, PostgreSQL).

- Supported bi-directional transfer: Import (RDBMS → HDFS/Hive/HBase) and Export (HDFS → RDBMS)
- Used MapReduce/YARN for parallel data transfer — efficient for large datasets
- Supported incremental loads (only new/updated rows) and full table imports
- Connector-based architecture with JDBC support for various databases

Limitations

- Batch-only — no real-time or streaming support
- High overhead for small datasets (MapReduce startup)
- Lacked built-in data validation
- No CDC (Change Data Capture) support

⚠ Retired (June 2021) — moved to Apache Attic

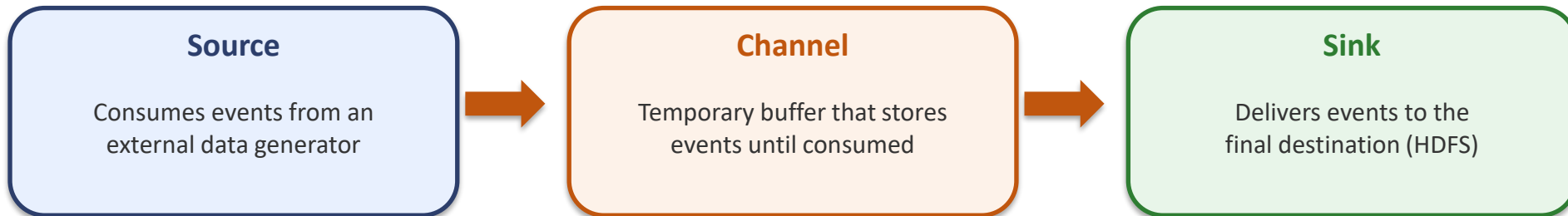
- SeaTunnel replaces Sqoop with 100+ connectors
 - Supports both batch AND streaming in one tool
 - Built-in CDC for real-time database sync
 - No MapReduce dependency — runs on Zeta/Flink/Spark
- Zeta is the default distributed engine for SeaTunnel

Apache Flume — A Legacy Tool

HADOOP ECOSYSTEM — DORMANT

Apache Flume is a distributed, reliable service for collecting, aggregating, and moving large volumes of streaming log data from many sources to a centralized store (typically HDFS).

Flume Agent Architecture



Limitations

- Streaming-only — no batch processing support
- No data transformation capabilities
- High latency; not suited for real-time analytics
- Dormant since October 2024 — no active maintenance

⚠ Dormant (Oct 2024) — not recommended for new projects

- SeaTunnel handles both streaming AND batch
- Built-in transform stage for in-flight reshaping
- 100+ connectors vs Flume's narrow log sources
- Actively maintained Apache Top-Level Project

What is Apache SeaTunnel?

Apache SeaTunnel is an easy-to-use, ultra-high-performance, distributed data integration platform that synchronizes massive volumes of data between heterogeneous systems, reliably and efficiently.

- Top-level Apache Software Foundation (ASF) project (graduated June 2023)
- Originally created as Waterdrop in 2017; renamed SeaTunnel in October 2021
- Entered the Apache Incubator in December 2021
- Unifies batch and streaming data movement in one tool
- Built to replace fragmented, hand-written sync scripts

100+

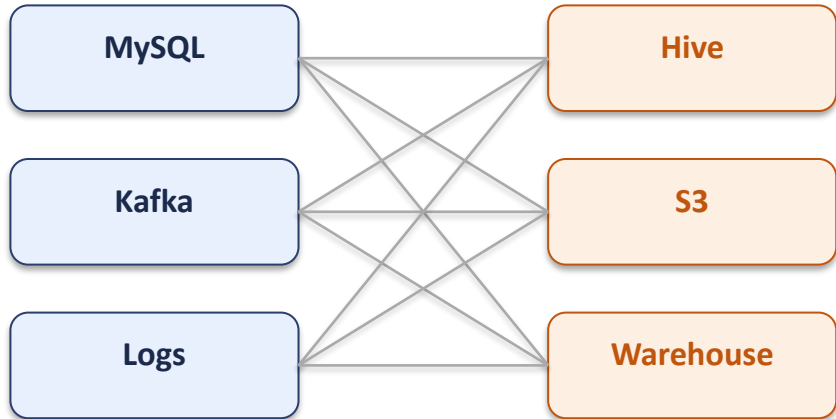
connectors across databases,
files, lakes & queues

1 Config File

HOCON / JSON / SQL — no Java
or Scala required

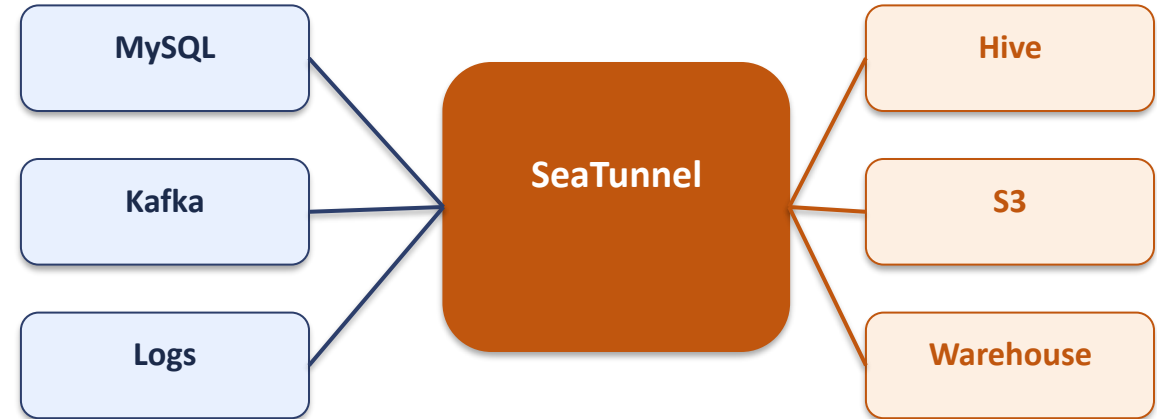
The $N \times M$ Connector Problem

BEFORE — Point-to-Point Scripts



9 custom point-to-point scripts to build & maintain

AFTER — Unified Integration Platform



3 + 3 = 6 connectors — write once, reuse anywhere

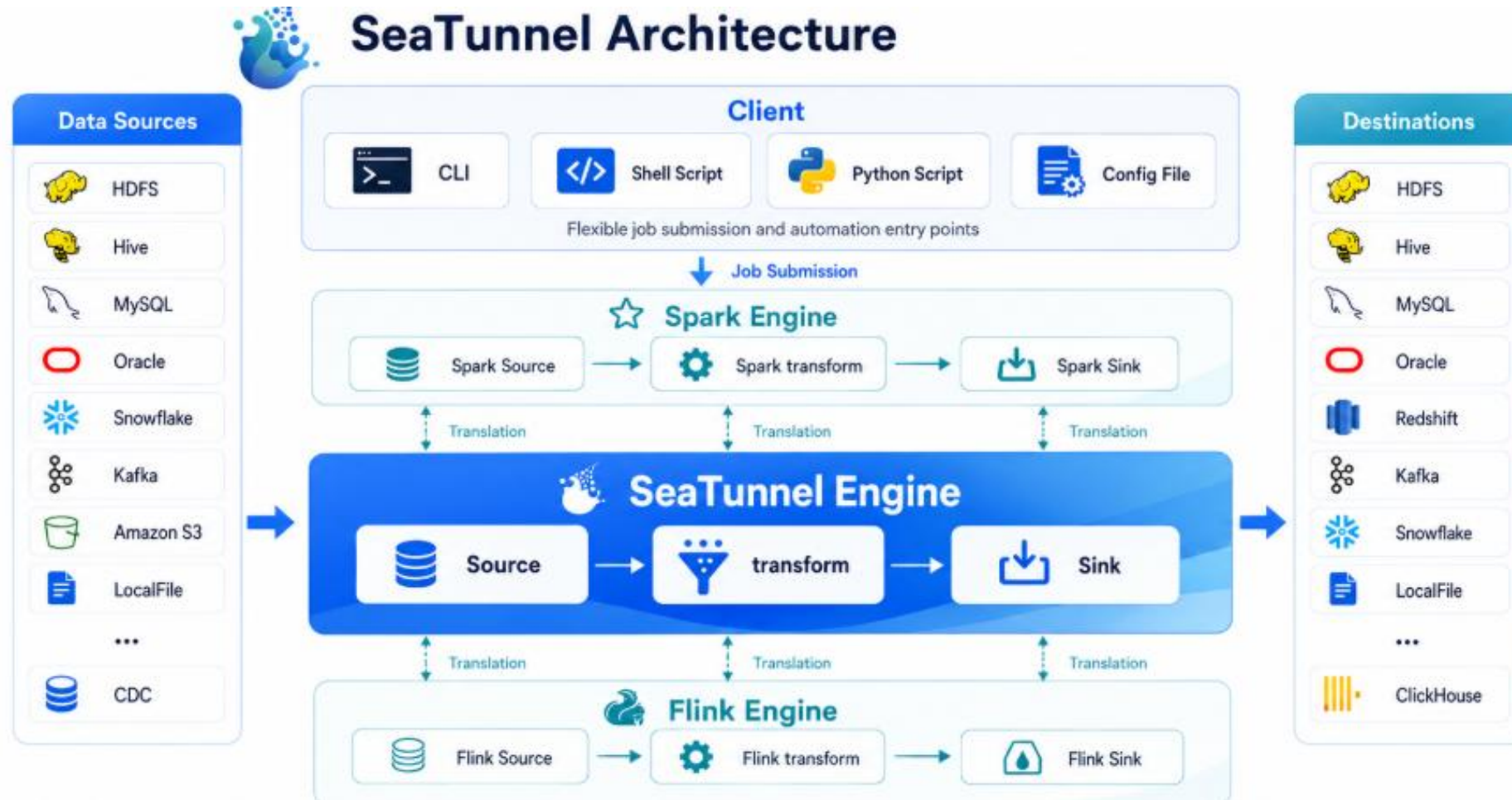
Key Features of Apache SeaTunnel

- **Batch & Streaming Support:** Use the same engine and configuration format to build both batch (offline) and streaming (real-time) data pipelines, simplifying development and maintenance.
- **Large Connector Library:** Connect easily to 100+ data sources and destinations — databases, files, data lakes, warehouses, message queues, and cloud/SaaS platforms.
- **Multiple Execution Engines:** Run the same SeaTunnel job on its built-in Zeta engine or on Apache Flink or Apache Spark, depending on performance and deployment needs.
- **Change Data Capture (CDC):** Continuously capture database changes (inserts, updates, deletes) for real-time synchronization without repeated full-table scans.
- **Exactly-Once Fault Tolerance:** Built-in checkpointing and distributed snapshots ensure data is never lost or duplicated, even during failures.
- **Low-Code Configuration:** Define complete data pipelines declaratively using HOCON, JSON, or SQL — no large amounts of custom code required.
- **Automated Schema Evolution:** Automatically detects and propagates upstream schema changes to downstream systems, preventing pipeline downtime.

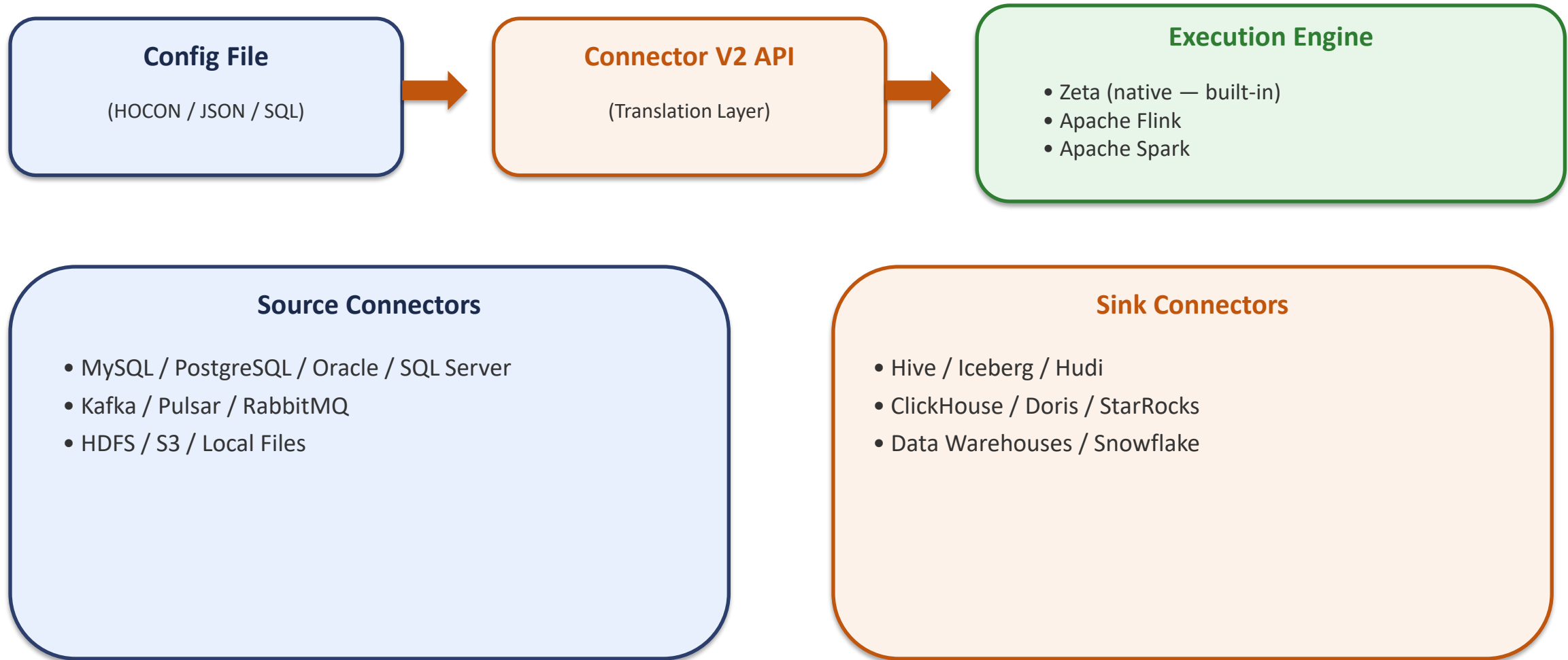
Pipeline Architecture: Source → Transform → Sink



How stages connect: each module tags its output with `plugin_output`; the next module reads it via `plugin_input`. If left unset, SeaTunnel simply feeds the previous module's output forward — like sections of pipe bolted end to end.



Architecture Deep Dive



1

The job is fully described in a config file — no code to compile.

2

The Connector V2 API decouples plugins from any one engine.

3

Pick the engine per job: lightweight Zeta, or an existing Flink/Spark cluster.

4

100+ connectors read and write on either side of the pipeline.

One Job, Three Possible Engines

Zeta

Native Engine

- Built into SeaTunnel — nothing else to install
- Lightweight; ideal for pure sync jobs
- Local, Hybrid & Separated cluster modes
- Recommended for most SeaTunnel deployments

Apache Flink

Streaming-First

- Reuses an existing Flink cluster
- Mature streaming semantics & state backend
- Good fit when Flink is already your standard
- Excellent for low-latency event processing

Apache Spark

Batch-First

- Reuses an existing Spark cluster
- Strong for large offline/batch workloads
- Familiar to teams already on Spark
- Best for heavy ETL transformations

The Config File — HOCON Format

A SeaTunnel job is one declarative file with up to four blocks. No Java, no Scala, no build step.

```
env {  
  job.mode = "BATCH"  
}  
  
source {  
  MySQL {  
    result_table_name = "orders"  
    table-name = "orders"  
  }  
}  
  
transform {  
  Filter {  
    source_table_name = "orders"  
    fields = [id, amount, city]  
  }  
}  
  
sink {  
  Hive { table = "orders_clean" }  
}
```

env

Global run settings — here, a one-time BATCH job (vs. STREAMING).

source

Where to read from. Every source has a result_table_name others can reference.

transform

Optional reshaping — here, keeping only 3 columns from the source data.

sink

Where the final rows land — in this case, a Hive table called 'orders_clean'.

Connectors & Change Data Capture (CDC)

Databases

MySQL · PostgreSQL · Oracle
SQL Server · TiDB · MongoDB

Lakes & Warehouses

Hive · Iceberg · Hudi · Doris
ClickHouse · StarRocks ·
Snowflake

File Systems

HDFS · S3 · OSS
Local FS · FTP

Message Queues

Kafka · Pulsar · RabbitMQ
RocketMQ · Amazon SQS

Change Data Capture (CDC)

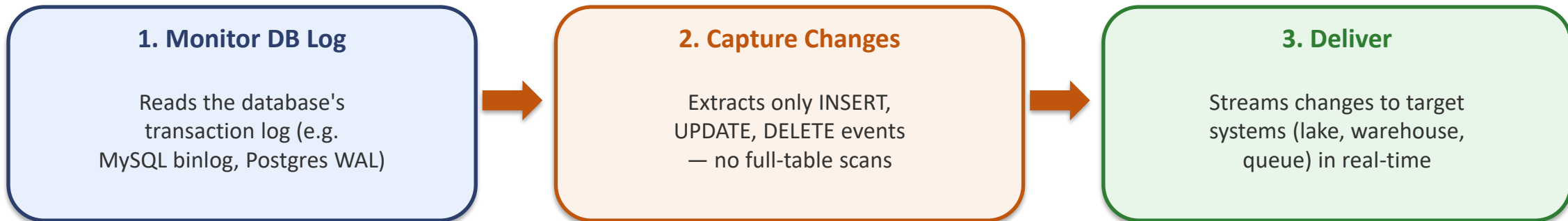
CDC connectors — MySQL-CDC, PostgreSQL-CDC, Oracle-CDC, MongoDB-CDC — read the database's own transaction log or oplog to stream every row-level change in real time, without extra load from repeated full-table queries.

This is the mechanism behind real-time data lake synchronization, and it is the biggest practical reason SeaTunnel fits modern near-real-time lake pipelines. The CDC approach avoids locking, minimizes source database impact, and supports exactly-once delivery semantics.

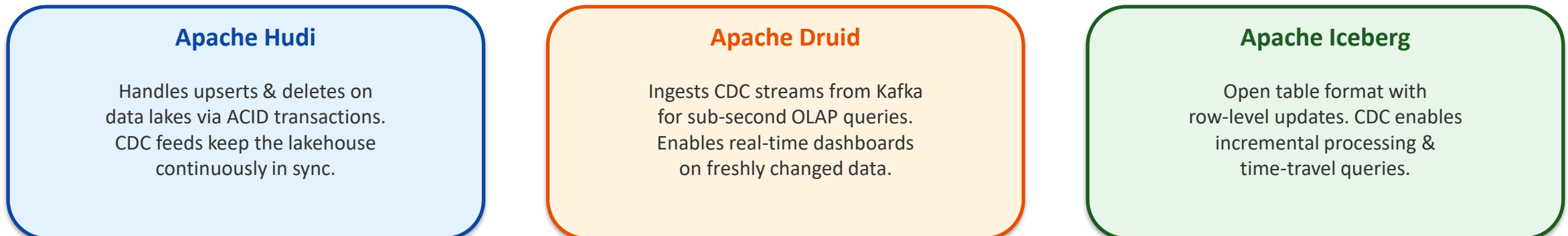
Change Data Capture (CDC) — Deep Dive

Change Data Capture (CDC) is a data integration pattern that identifies and tracks row-level changes — inserts, updates, and deletes — in a source database and delivers them to downstream systems in near real-time.

How CDC Works



Why CDC Matters — Foundation for Modern Data Tools



References & Sources

1. Apache SeaTunnel Official Documentation — seatunnel.apache.org/docs
2. Apache SeaTunnel GitHub Repository — github.com/apache/seatunnel
3. Apache Software Foundation — Attic: Sqoop — attic.apache.org/projects/sqoop.html
4. Apache Flume Project Status — flume.apache.org
5. "What the Heck is Apache SeaTunnel?" — hackernoon.com
6. Apache SeaTunnel: Top-Level Project Graduation Announcement — blogs.apache.org
7. SeaTunnel Architecture and Design Documentation — seatunnel.apache.org/docs/concept/architecture
8. Change Data Capture (CDC) Concepts — seatunnel.apache.org/docs/concept/connector-v2-features
9. Apache Hudi — hudi.apache.org | Apache Druid — druid.apache.org | Apache Iceberg — iceberg.apache.org



THANK YOU

Prafullata Kiran Auradkar

Department of CSE

prafullatak@pes.edu



PES
UNIVERSITY

CELEBRATING 50 YEARS

BIG DATA

Hadoop Ecosystem: Apache Airflow

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Content in this slide deck has been compiled from Apache Airflow official documentation and supplementary academic resources. Information may have been sourced from books, research papers, and internet references. Credit and rights remain with the original authors. These slides are intended for classroom presentation only.

What is a DAG? — Directed Acyclic Graph

DAG (Directed Acyclic Graph) is a mathematical structure used to model workflows as a set of tasks with explicit dependency relationships where no task can depend on itself directly or indirectly.

Three defining properties:

Directed

Edges carry a direction: Task A → Task B means B depends on A and must run only after A completes successfully.

Acyclic

No cycles are permitted. A task cannot depend on itself — directly or transitively. This guarantees a finite execution order.

Graph

Nodes represent tasks. Edges represent dependencies. The full structure captures the complete workflow topology.

Note: A DAG defines WHAT tasks to run and in WHAT ORDER — it does not execute tasks itself. The orchestrator (Airflow) is responsible for execution.

BIG DATA

DAG — Properties and Real-World Use Cases

Core Properties

Topological Ordering

Tasks can always be sorted so each appears before its dependents.

Parallel Execution

Independent tasks with no shared dependency run simultaneously.

Deterministic Execution

Given the same inputs, a DAG always produces the same task order.

Fault Isolation

A failed task can be retried in isolation without re-running the pipeline.

Idempotency

Workflows can be safely re-triggered: rerunning produces the same result.

Real-World Workflows as DAGs

Data ETL Pipeline

Extract > Transform > Validate > Load > Notify

ML Training Pipeline

Pre-process > Train > Evaluate > Register Model

CI/CD Pipeline

Build > Unit Test > Integration Test > Deploy

Report Generation

Query DB > Aggregate > Format > Email > Archive

Apache Oozie — Legacy Hadoop Workflow Scheduler

HADOOP ECOSYSTEM — LEGACY ORCHESTRATOR

Apache Oozie is a server-based workflow scheduler system for Hadoop jobs. It coordinates MapReduce and Pig jobs using XML-based workflow definitions, enabling multi-step data processing pipelines on a YARN cluster.

Core Capabilities

- Workflow Jobs: DAG of actions — MapReduce, Hive, Pig, Shell, Java
- Coordinator Jobs: time/data-triggered recurring workflow execution
- Bundle Jobs: grouping and coordinating multiple coordinators
- Tight YARN/HDFS integration — built for on-premise Hadoop clusters
- Web console for monitoring, rerunning, and killing jobs

Oozie Architecture

Oozie Server

Central service coordinating execution of all jobs

Workflow Engine

Executes the workflow DAG step by step

Coordinator Engine

Triggers workflows on time or data availability

Metadata DB (Derby/MySQL)

Stores all workflow state, history, and metadata

Apache Oozie — Limitations and Why Airflow Replaced It

Oozie served Hadoop clusters well in the MapReduce era. As data engineering evolved beyond HDFS-centric batch processing, its structural limitations became blockers.

Limitations of Oozie

- XML-based workflows: verbose, hard to version-control, error-prone
- No Python support: only Java actions or Shell scripts
- Tightly coupled to Hadoop/YARN/HDFS — no cloud-agnostic design
- No dynamic task generation: static DAGs only
- Poor debugging experience: limited failure visibility
- No support for modern stores: S3, BigQuery, Snowflake
- Slow community innovation: in maintenance mode since 2020
- Basic UI: minimal monitoring and alerting capabilities

Why Airflow has the edge

- Python-native: workflows defined as Python code (DAGs as code)
- Dynamic DAGs: pipelines generated programmatically at runtime
- Rich operator library: 1000+ providers across cloud and on-prem
- Modern web UI: rich monitoring, live logs, task retries, Gantt view
- Active community: top-10 Apache project by contribution volume
- Cloud-agnostic: AWS, GCP, Azure, Kubernetes, on-prem
- Pluggable executors: Local, Celery, Kubernetes, hybrid
- SLA monitoring, alerting, XCom for cross-task communication

What is Apache Airflow?

HADOOP ECOSYSTEM — ACTIVE / CLOUD-NATIVE

Apache Airflow is an open-source platform to programmatically author, schedule, and monitor workflows. Workflows are defined as Directed Acyclic Graphs (DAGs) written in Python. Airflow originated at Airbnb in 2014 and became an Apache Top-Level Project in 2019.

Founded Airbnb, 2014 Apache TLP, 2019	Language Python 3.x (Pure Python DAGs)	License Apache 2.0 Open Source	Version Airflow 2.x / 3.x (3.3+ in 2026)
--	---	---	---

Philosophy: Configuration as Code

- Workflows are Python scripts — versioned, testable, peer-reviewed
- Separates workflow logic from execution infrastructure
- Not a data streaming framework — it is an orchestrator
- Motto: "Author, Schedule, Monitor"

Ecosystem Scale (2025)

30,000+ GitHub Stars	1,500+ Contributors
70+ Cloud integrations	10M+ Daily DAG runs (est.)

Why Apache Airflow? — The Orchestration Problem

Before Airflow, teams relied on cron jobs, custom shell scripts, and legacy tools that could not meet the reliability, observability, and scalability demands of modern data engineering.

Traditional Approach	Airflow Solution
No retry on failure	Configurable retry policy per task with exponential back-off
No dependency management	Explicit task dependencies expressed directly in the DAG
No pipeline visibility	Real-time UI with logs, Gantt chart, and task state history
Single-machine bottleneck	Celery/Kubernetes Executor for distributed, scalable execution
Hard-coded cron schedules	Cron, event-driven (Dataset), manual, or API-triggered runs
Config in XML or shell scripts	DAGs as Python code: testable, reviewable, version-controlled

Apache Airflow — Core Architecture

Airflow follows a modular, distributed architecture. Each component has a distinct responsibility, enabling horizontal scaling and fault tolerance.

Web Server

FastAPI/Gunicorn HTTP server serving the Airflow UI. Allows users to trigger DAGs, inspect logs, manage connections and variables.

Scheduler

Heart of Airflow. Continuously scans DAG files, evaluates task dependencies, and submits ready tasks to the Executor.

Executor

Determines HOW and WHERE tasks run: LocalExecutor (single node), CeleryExecutor (distributed), KubernetesExecutor (pods).

Metadata Database

PostgreSQL or MySQL store for DAG run state, task instances, connections, variables, XComs, and SLA records.

Worker(s)

Processes that execute task code. With CeleryExecutor, workers pull tasks from a message queue (Redis or RabbitMQ).

DAG Processor

Parses Python DAG files on a regular cycle and registers them in the metadata database. Runs as a separate process from Airflow 2.x onward.

Airflow DAG: Python-Based Workflow Definition

An Airflow DAG is a Python script. The developer instantiates a DAG object, defines Operators as tasks, and sets dependencies using the ">>" operator. The scheduler reads these files and executes them on schedule.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract(): ...
def transform(): ...
def load(): ...

with DAG(
    dag_id='etl_pipeline',
    start_date=datetime(2025, 1, 1),
    schedule='@daily',
    catchup=False,
) as dag:

    t1 = PythonOperator(task_id='extract', python_callable=extract)
    t2 = PythonOperator(task_id='transform',
python_callable=transform)
    t3 = PythonOperator(task_id='load', python_callable=load)

    t1 >> t2 >> t3
```

DAG Object

The 'with DAG()' context manager sets the dag_id, start_date, and schedule. All operators inside the block are registered as tasks.

Operators

PythonOperator, BashOperator, SQLOperator etc. Each operator becomes one node (task) in the DAG graph.

Dependencies >>

t1 >> t2 >> t3 runs tasks sequentially. Independent tasks set separately run in parallel.

schedule=

Accepts cron strings ('@daily', '0 6 * * *'), timedelta, Dataset objects (event-driven), or None for manual-only.

Airflow Operators — Building Blocks of Tasks

An Operator is a template for a single task unit. Airflow ships with 40+ built-in operators and 1000+ additional operators via provider packages, covering virtually every data platform and cloud service.

Core Operators

PythonOperator
BashOperator
BranchPythonOperator
EmptyOperator

Database Operators

PostgresOperator
MySqlOperator
SqliteOperator
BigQueryOperator

Cloud: AWS

S3CopyObjectOperator
GlueJobOperator
EMRCreateJobFlowOperator

Cloud: GCP

GCSToBigQueryOperator
DataprocJobOperator
CloudRunOperator

Cloud: Azure

AzureBlobStorageOperator
ADFRunPipelineOperator
SynapseRunSparkJobOperator

ML and Spark

SparkSubmitOperator
DatabricksRunNowOperator
SageMakerTrainingOperator

Providers are pip-installable: `pip install apache-airflow-providers-google`

Apache Airflow — Key Features

Dynamic Pipelines

DAGs are Python code: generate tasks dynamically using loops, conditions, and data fetched from external sources at parse time.

Extensible

Custom operators, hooks, sensors, and executors. Plugin system allows organisation-specific extensions without forking Airflow.

Rich Web UI

Graph, Grid, Gantt, and Tree views of DAG runs. Live log streaming, retry controls, variable management, all from the browser.

Flexible Scheduling

Cron, timedelta, Dataset-trigger (event-driven), or manual. Backfill support enables reprocessing of historical date ranges.

XCom Communication

Tasks share small data values via XCom (cross-communication). Push/pull mechanism stored in the metadata database.

Sensors

Special wait-operators that block until an external condition is met: file arrival, API response, SQL row count threshold, S3 object existence.

SLA and Alerting

Define Service Level Agreements per task. Airflow sends email or Slack alerts automatically on SLA misses or task failures.

Connections and Variables

Centralised credential store for DB connections and API keys. Runtime Variables provide config injection without hard-coding values in DAGs.

Apache Airflow — Executors and Scaling Strategies

The Executor determines how and where task instances are executed. Choosing the right executor is critical for performance, cost, and operational complexity.

Sequential Executor

- Development only
- One task at a time
- No parallelism
- SQLite backend
- Not for production

Local Executor

- Small teams / single node
- Subprocess parallelism
- Needs PostgreSQL
- Simple setup

Celery Executor

- Distributed scale-out
- Worker pool via Redis
- Horizontally scalable
- Production-grade

Kubernetes Executor

- Cloud-native / serverless
- One K8s pod per task
- Auto-scales to zero
- Best isolation

CeleryKubernetes Executor

- Hybrid approach
- Celery for fast tasks
- K8s for heavy tasks
- Flexible resources

Development

Production / Enterprise

Apache Airflow — Scheduling, Triggers and Backfill

The Airflow Scheduler supports multiple trigger strategies beyond simple cron expressions — enabling time-driven, event-driven, data-driven, and manual orchestration.

Scheduling Strategies

Cron Expression

```
schedule='0 6 * * *'
```

Runs every day at 06:00 UTC. Standard cron syntax. Human-readable aliases: @daily, @hourly, @weekly, @monthly.

Timedelta

```
schedule=timedelta(hours=6)
```

Runs every 6 hours relative to the previous run. Useful for offset scheduling patterns.

Dataset Trigger (Airflow 2.4+)

```
schedule=Dataset('s3://bucket/file.csv')
```

Run when an upstream DAG produces or updates a Dataset. Enables true data-driven pipelines without polling.

Manual / None

```
schedule=None
```

Triggered via the UI, CLI (airflow dags trigger), or REST API only. Common for on-demand or ad-hoc pipelines.

Backfill and Catchup

catchup=True (default)

If a DAG has a past start_date, Airflow schedules all missing historical runs automatically. Useful for reprocessing historical data.

catchup=False

Only run from the current time onwards. Prevents flooding the scheduler with hundreds of backfill runs on first deployment.

CLI Backfill

```
airflow dags backfill \  
  --start-date 2025-01-01 \  
  --end-date 2025-01-31 \  
  my_dag  
Manually trigger historical date-range runs.
```

Apache Airflow — Web UI and Monitoring

The Airflow Web UI (port 8080) is the primary operational interface for DAG management, monitoring, debugging, and control. Built on Flask/FastAPI with a React frontend.

DAGs List View

All DAGs with schedule, last run time, success/failure counts. Toggle pause/unpause.

Graph View

Interactive DAG topology. Click any task to inspect logs, state, and retry options.

Grid View

Matrix of DAG runs vs tasks. Color-coded states: green success, red failed, yellow running.

Gantt View

Timeline of task durations. Identify bottlenecks and parallelism opportunities.

Task Logs

Full stdout/stderr per task instance. Live-streamed during execution. Searchable.

XComs Browser

Inspect values passed between tasks. Useful for debugging data flow through the pipeline.

Connections

Manage credentials for databases, APIs, and cloud services. Encrypted with Fernet.

Variables

Runtime key-value config store. Retrieve in DAGs via `Variable.get('key')`.

Apache Airflow — Real-World Case Studies

Airflow originated at Airbnb and is now deployed by hundreds of data-driven organisations worldwide — from startups to hyperscalers.

Airbnb

Use Case

Core ETL orchestration: booking/search events to data warehouse. 1000+ DAGs, 100K+ task instances per day.

Outcome

Replaced cron and internal tooling. Engineers iterated faster. Pipeline failures detected within minutes via alerting.

Spotify

Use Case

Recommendation model training: ingest user events, feature engineering, train, evaluate, deploy. KubernetesExecutor, 500+ DAGs.

Outcome

Daily model refresh cycle. Each task isolated in a Kubernetes pod. No shared-environment dependency conflicts.

LinkedIn

Use Case

Feed ranking and analytics processing billions of events per day. Data quality gates block downstream tasks on failure.

Outcome

SLA monitoring reduced data latency incidents by ~40%. Backfill critical for A/B test re-analysis.

NASA JPL

Use Case

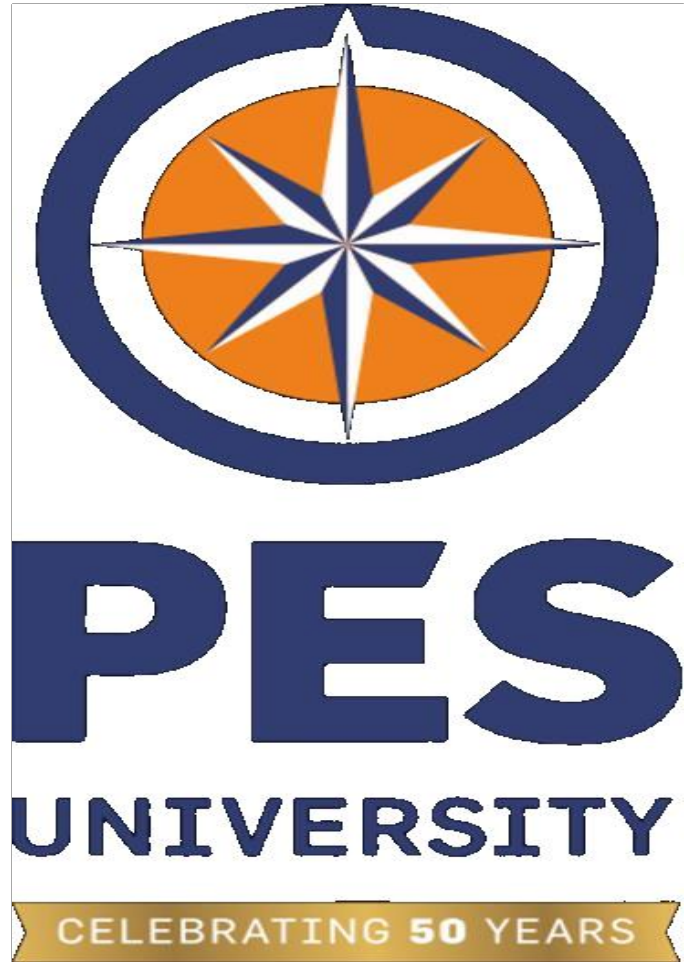
Scientific data pipelines: ingest planetary sensor readings, transform, store, alert mission scientists.

Outcome

Sensor operators poll for data arrival before processing. Retry and alerting ensure zero missed science data events.

References and Sources

1. Apache Airflow Official Documentation -- airflow.apache.org/docs
2. Apache Airflow GitHub Repository -- github.com/apache/airflow
3. Apache Oozie Official Documentation -- oozie.apache.org
4. Apache Software Foundation -- Attic: Oozie -- attic.apache.org/projects/oozie.html
5. "Airflow: A Platform to Program Workflows at Airbnb" -- Maxime Beauchemin, 2015
6. Apache Airflow: Top-Level Project Graduation Announcement -- blogs.apache.org
7. Airflow Architecture and Concepts -- airflow.apache.org/docs/apache-airflow/stable/concepts
8. Directed Acyclic Graphs in Data Engineering -- towardsdatascience.com
9. Managing Cloud Composer -- cloud.google.com/composer/docs
10. Amazon MWAA Documentation -- docs.aws.amazon.com/mwaa



THANK YOU

Prafullata Kiran Auradkar

Department of CSE

prafullatak@pes.edu



BIG DATA

Data Lake and Lakehouse, Open Table Formats and Apache Iceberg

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Content in this slide deck has been compiled from Apache official documentation and supplementary academic resources. Information may have been sourced from books, research papers, and internet references. Credit and rights remain with the original authors. These slides are intended for classroom presentation only.

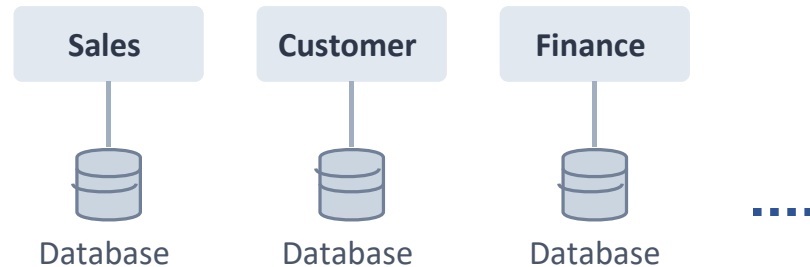
- Data Lakehouse
 - Evolution from Data Warehouse to Data Lake to Data Lakehouse
- Open Table Format
- Apache Iceberg
- Source
 - Shiran, Tomer. Apache Iceberg: The Definitive Guide: Data Lakehouse Functionality, Performance, and Scalability on the Data Lake. O'Reilly Media.
 - <https://hudi.apache.org/blog/2024/07/11/what-is-a-data-lakehouse/>
 - <https://www.databricks.com/discover/data-lakes>

BIG DATA

From Databases to Data Warehouses to Data Lake

Databases

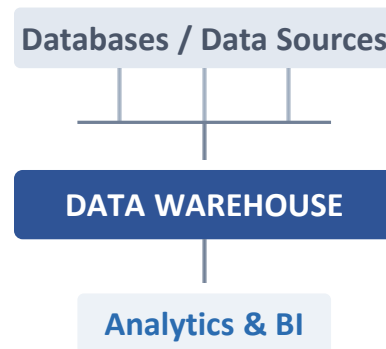
Store and manage data for individual applications and operational systems.



As organizations grow, data becomes distributed across many systems.

Data Warehouse

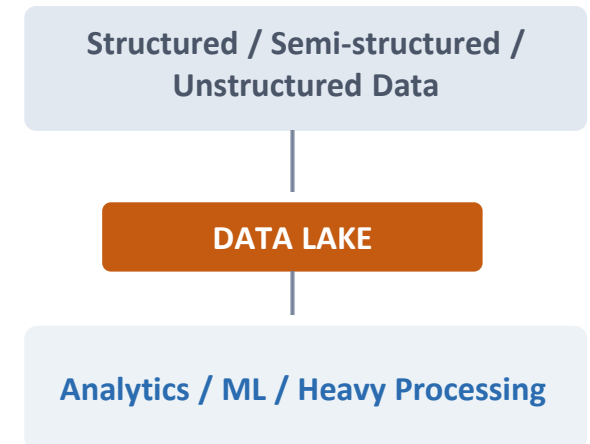
Consolidate and structure data from multiple sources together for large-scale business intelligence.



Warehouses traditionally work best with **structured**, prepared data and can become **expensive/less flexible** as data **volume** and **variety** grow.

Data Lake

Store massive amounts of highly diverse data in its raw, original form on highly scalable storage (on-prem HDFS or cloud based).



Analytics / ML / Heavy Processing

The data lake solves the storage problem:

- Data can be stored at massive scale, in its original form, without requiring everything to be structured upfront. (low cost)
- The same stored data can be used for different analytical workloads. (flexible)
- Different compute engines can be used.

But storing data is not enough.

To use the data effectively, we also need to:

- Organize and manage large collections of data
- Efficiently query and process the data
- Reliably update and maintain data over time

A data lake gives us flexible, scalable storage but does not by itself provide all the capabilities we expect from a well-managed analytical data system.

Comparison	Data Warehouse	Data Lake
Data	Primarily structured data	Structured, semi-structured, unstructured data
When is structure defined?	Schema-on-write structure is defined before data is stored	Schema-on-read data can be stored first and interpreted when used
Storage	Generally more expensive	Lower-cost , scalable storage
Data formats	Physical storage format is managed by the warehouse system	Can store data in various file formats, including CSV, JSON, Parquet, ORC, Avro
Compute	Typically tied to the warehouse system	Different compute engines can be used for different workloads
Management/ performance	More managed and optimized for analytical queries	More flexible , but requires more configuration and optimization

A Data Lakehouse combines the strengths of a Data Lake and Data Warehouse:

Data Lake

- scalable, lower-cost storage
- open formats
- diverse data
- multiple compute engines

Data Warehouse

- reliable data management
- good analytical performance
- easy updates and querying

How?

Keep the data, storage and file formats of the data lake, but add warehouse-like table capabilities through a modern **open table format**.

Note: *The lakehouse does not replace the data lake, it adds the capabilities needed to use lake data as reliable analytical tables.*

How does a Data Lake become a Lakehouse?

A data lake stores data as files. But analytics needs data to behave like reliable, structured tables:

File Mapping: Which files belong to a table?

Schema Definition: What is the table's schema?

Data Mutation: How do we safely add, update or delete data?

Version Control: How do we keep track of different versions?

The Open Table Format Solution

- Provides the table-management layer on top of data-lake storage. (Ex: Apache Iceberg, Hudi)
- **Why "Open"?** The table format is an **open specification**, not tied to one storage system or compute engine.
- *Note: Does not replace underlying storage; it manages how files are treated as tables.*

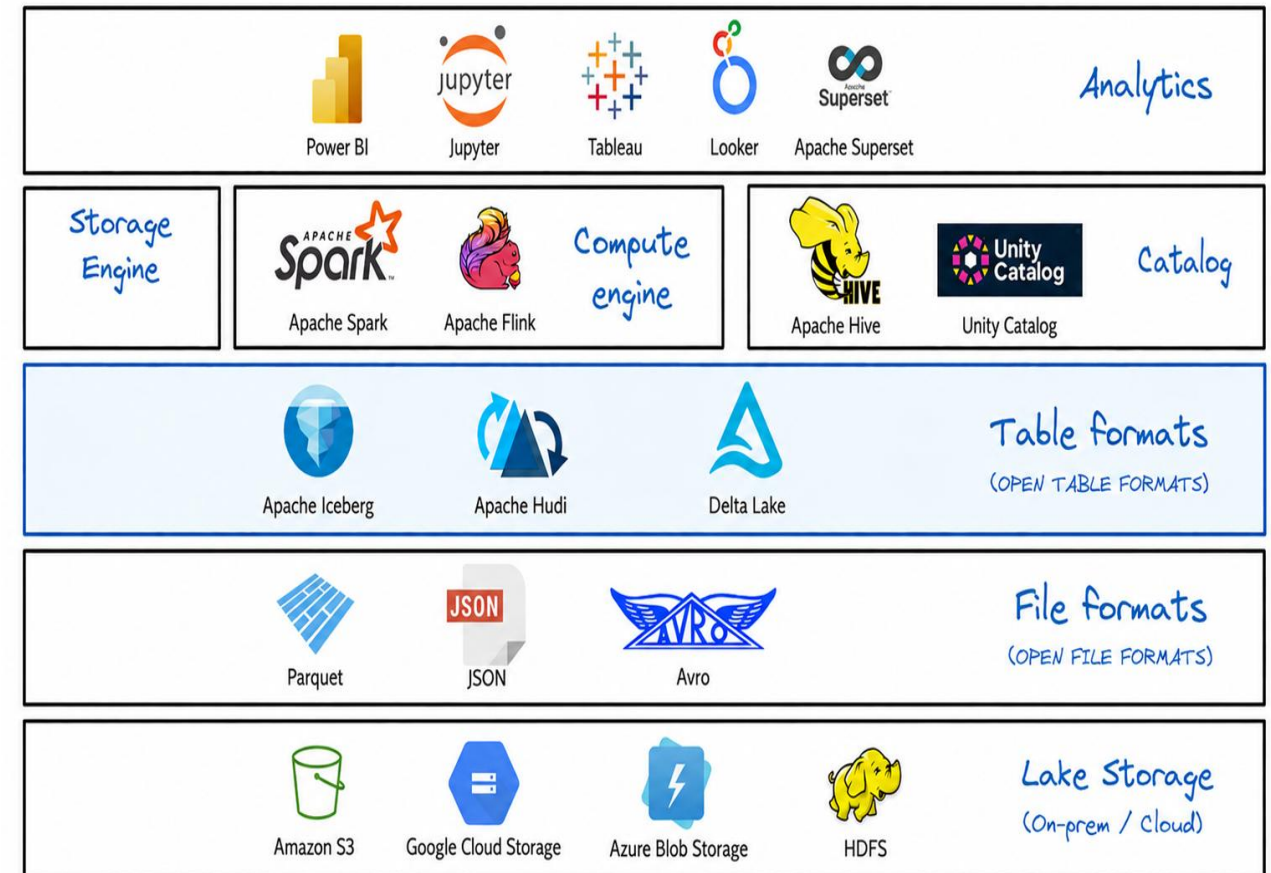


Figure: Data Lakehouse Architecture

BIG DATA

Apache Iceberg

- **Created by Netflix in 2017** to address limitations of the Hive table format for very large analytical tables.
- **Open-sourced in 2018** and donated to the Apache Software Foundation.
- Designed around three goals:
 - **Consistency** - reliable, atomic table changes
 - **Performance** - efficient query planning without excessive file listing
 - **Ease of use** - users should not need to know the table's physical layout
- Apache Iceberg is an **open table format** for huge analytic datasets.
- Iceberg **does not provide storage or a compute engine**, it enables existing engines to work with tables stored in the data lake.



An Iceberg table has three layers: Catalog, Metadata, and Data.

1. Catalog - Find the table

Catalog → Points to the table's current metadata file

2. Metadata - Describe and track the table

Metadata file → Describes the table: schema, partitioning, snapshots

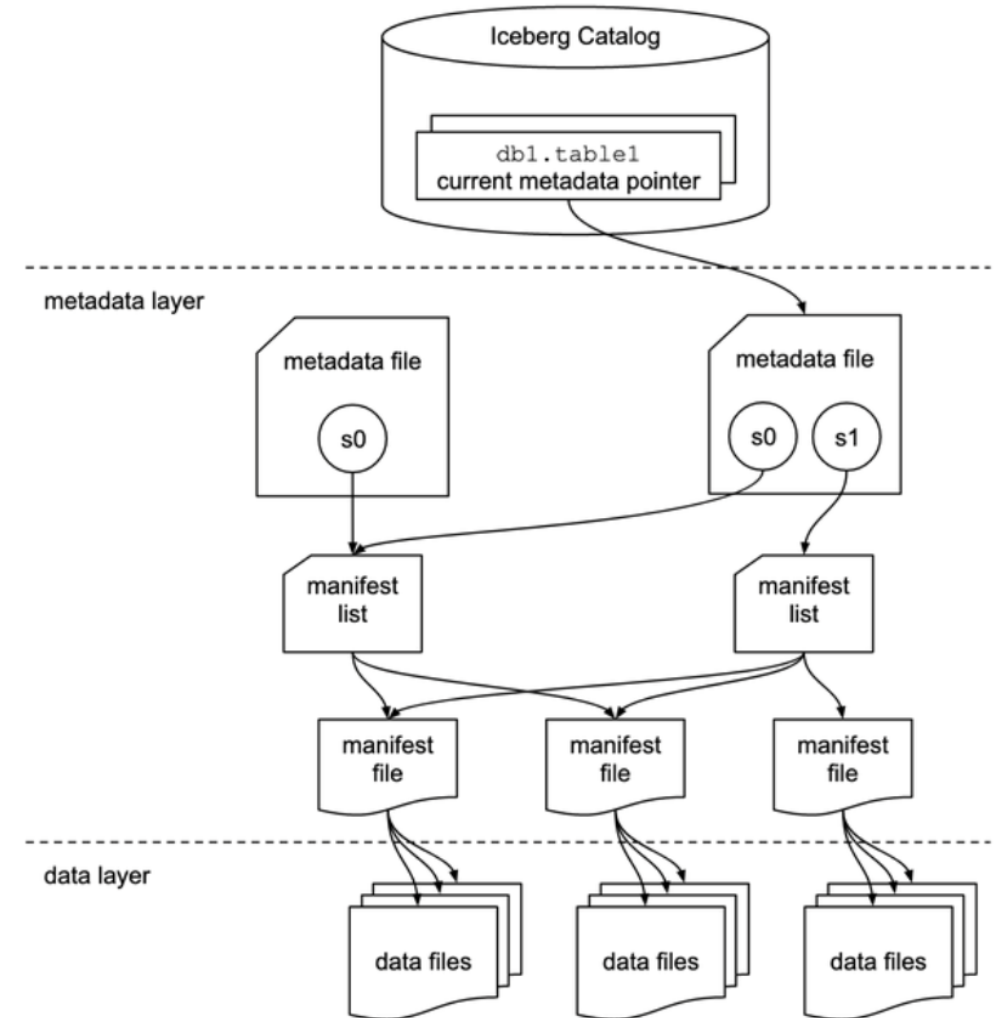
Snapshot → Represents the state of the table at a point in time

Manifest list → Identifies the manifest files for a snapshot

Manifest files → List the table's data files and information about those files

3. Data - Store the actual data

Data files → Contain the actual table data (e.g., Parquet)



01

Row-Level Operations

Supports updates and deletes via Copy-on-Write (COW) and Merge-on-Read (MOR) approaches.

02

Hidden Partitioning

The partitioning is fixed by the table's partition specification, but it is hidden from the user's logical view.

03

Partition Evolution

The partitioning can evolve without rewriting existing data.

04

Snapshots

Enable seamless time travel queries and quick rollbacks to previous committed states of tables.

05

Schema Evolution

The table schema can change without rebuilding all existing data.

06

Reliable Updates

ACID transaction support guarantees concurrent writers do not corrupt table consistency.

1. Row-Level Operations

Iceberg supports changes to individual rows, like **UPDATE** (modify existing) and **DELETE** (remove existing).

Copy-on-Write (COW)

When a row is changed, the entire data file containing that row is rewritten with the updated data.

$$[A \ B \ C \ D] \rightarrow [A \ B \ C' \ D]$$

Trade-off: **More data written** ; simpler, faster reads

Merge-on-Read (MOR)

The change is written separately to a new file and merged with the original data file during read time.

$$[A \ B \ C \ D] + [C \ C']$$

↓
read/merge

Trade-off: **Less data written** ; additional work/latency during reads

Key Idea: COW and MOR are complementary strategies, allowing you to trade write latency against read performance depending on your workload requirements.

2. Hidden Partitioning

Iceberg stores the partition specification in metadata, hiding the physical layout from the user's logical view.

With Hidden Partitioning

Users query the logical column directly. Iceberg automatically maps it to physical partitions behind the scenes.

```
SELECT * FROM events
WHERE event_timestamp >= '2026-08-10'
AND event_timestamp < '2026-08-11'
```

Benefit: No need to know physical columns like `day(event_timestamp)`.

Without Hidden Partitioning

Physical partition columns are exposed. Users must manually add them to queries to enable partition pruning.

```
SELECT * FROM events
WHERE event_year = 2026 AND event_month = 8
AND event_day = 10
AND event_timestamp >= '2026-08-10'...
```

Drawback: Queries are complex, tightly coupled, and break if partitioning changes.

Key Advantage: Decoupling the physical layout from user queries allows database administrators to evolve partitioning strategies without breaking existing application queries.

3. Partition Evolution

As data grows, the physical partitioning strategy may need to change.

With Partition Evolution

The partitioning can evolve without rewriting existing data. Queries continue to use the same logical column; Iceberg uses the appropriate partition information for each part of the table.

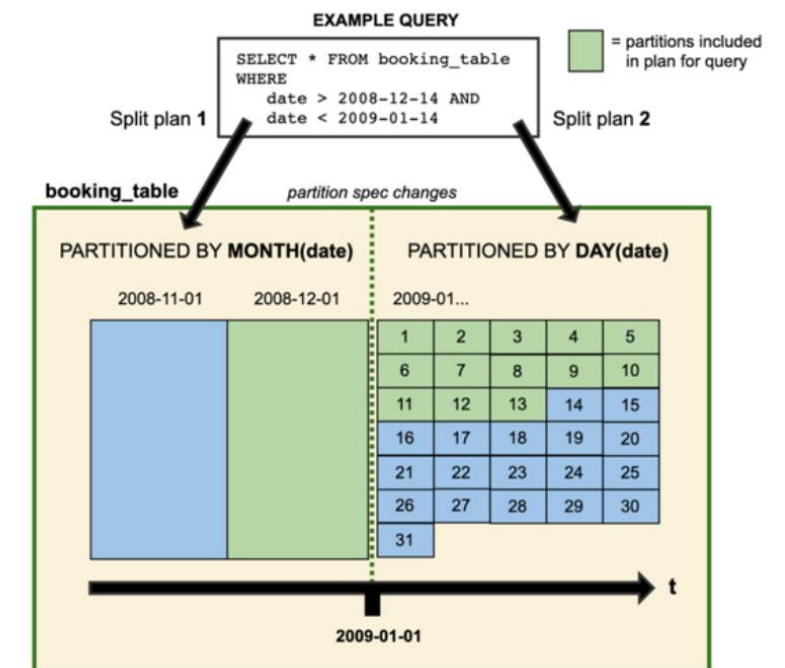
Older data → **PARTITION BY month(date)**

Newer data → **PARTITION BY day(date)**

Without Partition Evolution

Changing a table's partitioning would typically require rewriting/repartitioning the existing data, which can be very expensive at large scale.

Alternatively, you would keep the old partitioning and sacrifice the performance benefits of the new scheme.



4. Snapshots

Each committed table state is recorded as a snapshot.

Example Scenario

Snapshot 1 (1000 orders) → +50 new orders → Snapshot 2 (1050 orders)

Later, you discover that the 50 orders added in Snapshot 2 were incorrect.

Time Travel

Query the table as it existed at Snapshot 1.

```
SELECT * FROM table AS OF snapshot_1
```

Rollback

Restore the table's current state to Snapshot 1.

```
ROLLBACK TABLE table TO snapshot_1
```

Snapshots let us access and restore previous table states without maintaining a separate copy of the table.

5. Schema Evolution

The table schema can change without rebuilding all existing data.

Before

```
orders(order_id, customer_id,  
amount)
```

→ add column

After

```
orders(order_id, customer_id, amount,  
discount)
```

Existing records can remain as they are; the table's schema and metadata evolve to accommodate the new column. The table can evolve as requirements change without rewriting the entire dataset.

6. Reliable Updates

Table changes are committed atomically, so readers see a consistent table state.

Example: Suppose we update 1 million orders

Without transactional table management, a reader could potentially encounter a partially completed update.

With Iceberg:

Before update → Update in progress → After commit

State A

State B



reader sees A

reader sees B

Readers see either the table before the update or after the update — not a partially updated table.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Apache Hudi

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

`prafullatak@pes.edu`

Acknowledgements:

Content in this slide deck has been compiled from Apache official documentation and supplementary academic resources. Information may have been sourced from books, research papers, and internet references. Credit and rights remain with the original authors. These slides are intended for classroom presentation only.

- A data lake is storage infrastructure, but the data itself still needs to be represented in files.

For example, let's look at a simple table

order_id	customer	Status	update_at
1001	Alice	PENDING	1000
1002	Bob	PAID	1000
1003	Charlie	SHIPPED	1000

At the logical level, we think of this as a table of rows and columns. At the physical level, a data lake stores data as files.

Now one storage format for this is:

Apache Parquet:

Parquet is a column-oriented file format designed for efficient analytical workloads.

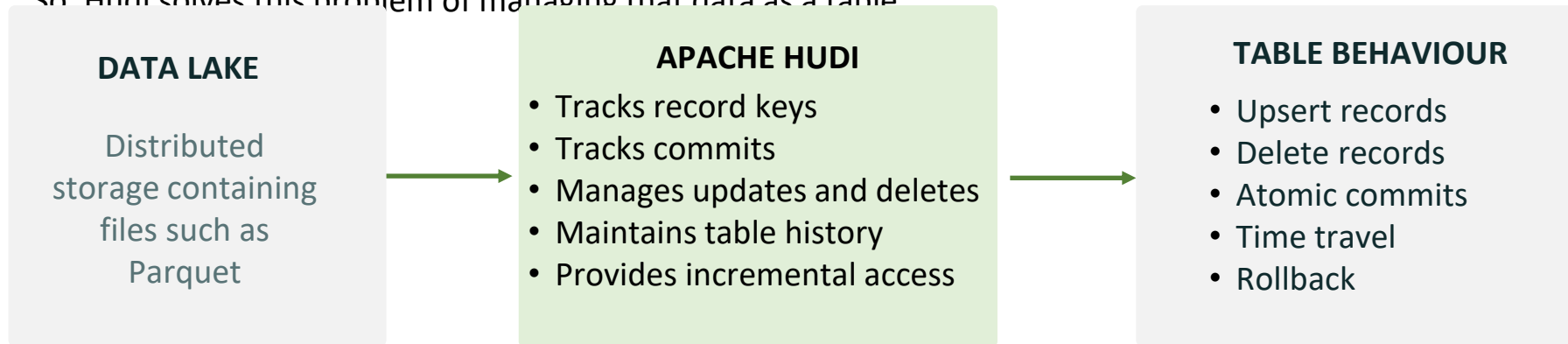
So, Parquet answers the question of how data should be stored efficiently.

The table to the left could be stored as:

```
orders/  
├─ part-00000.parquet  
├─ part-00001.parquet  
└─ part-00002.parquet
```

But Parquet is a file format, not a database system. It does not define how to update a single record while maintaining consistency or history.

So, Hudi solves this problem of managing that data as a table



Hudi does not replace the data lake. It adds a layer that gives the files table semantics.

For Hudi to update or delete an individual record, it must be able to uniquely identify that record.

Consider the same table,

Record Key: Hudi uses a record key to identify individual records within a table

For example, recordKey = order_id

The concept is similar to a primary key in a relational database, but Hudi uses the record key specifically for identifying records during write operations.

order_id	customer	Status	update_at
1001	Alice	PENDING	1000
1002	Bob	PAID	1000
1003	Charlie	SHIPPED	1000

When multiple versions of the same record key arrive in a single write, Hudi needs a way to determine which version represents the latest record.

For example, When two records have the same `order_id`, Hudi needs a **merge rule** to determine which record represents the table's current state.

order_id	customer	Status	update_at
1001	Alice	PENDING	1000
1002	Bob	PAID	1000
1003	Charlie	SHIPPED	1000

Hudi 1.0.2 provides three merge modes:

COMMIT_TIME_ORDERING - record from the later Hudi commit wins

EVENT_TIME_ORDERING - record with the greater preCombineField value wins

CUSTOM - A user-provided merger defines the rule

For `order_id = 1001`, if the incoming records have `update_at = 500` and the current `update_at = 2000`, **EVENT_TIME_ORDERING** selects the 2000 record because the greater preCombineField value wins. **COMMIT_TIME_ORDERING** selects the 500 because it is the later commit.

Hudi supports operations that modify the current state of a table while maintaining its commit history.

1. Insert

Adds a new record whose record key does not already exist:

1004	Dave	PLACED	1000
------	------	--------	------

2. Upsert

Inserts a record if its key does not exist, otherwise, Hudi merges it with the existing record according to the configured merge rules.

1001	ALICE	COMPLETED	2000
------	-------	-----------	------

3. Delete

Removes a record from the current table state while preserving the operation in Hudi's commit history.

Hudi supports two table types that differ in how updates are physically stored.

Copy-on-Write (CoW)	Merge-on-Read (MoR)
• Updates modify the existing base data files • A new version of the affected file is written containing the updated records • Reads are simpler because the latest data is already in the base files	• Updates are initially written to delta log files instead of immediately rewriting the base files • During a read, Hudi can combine the base files with the log files to produce the latest table state
Tradeoff: updates can require more data to be written	Tradeoff: writes are cheaper, but reads may require additional merging

Both COW and MOR provide the same table-level features; they differ primarily in how Hudi stores updates and serves reads.

Every change to a Hudi table is recorded as an event on the Hudi timeline. The timeline provides the table's history and allows Hudi to determine which table state is currently valid.

What is a commit?

- A commit represents a completed write operation on the table
- **A commit $\neq$ a changed row.** A commit represents a completed write operation. So, it doesn't necessarily mean every incoming record changed the table's final state.
- Each commit receives a unique **commit time**.
- The timeline is stored in the table's **.hoodie metadata directory**.
- Hudi uses the timeline to determine the sequence of table changes and the valid state of the table.

```
scala> spark.sql("CALL show_commits(table => 'orders', limit => 20)").select("commit_time", "action", "total_records_written", "total_update_records_written").show(false)
+-----+-----+-----+-----+
|commit_time|action|total_records_written|total_update_records_written|
+-----+-----+-----+-----+
|20260809230417616|commit|3|1|
|20260809230210361|commit|3|0|
+-----+-----+-----+-----+
```

Time Travel

- The timeline contains the history of the table, so Hudi can reconstruct the table as it existed at an earlier point in time.
- **Time travel** allows a table to be queried using a previous **commit instant** rather than its current state.
- Time travel does not undo or modify the table. It only changes which point in the table's history is being read. It is simply a snapshot of the table at that commit.

Incremental Queries

- Hudi's timeline can be used not only to recover previous table states, but also to identify changes made after a particular point in time.
- Reads records associated with commits **after a specified commit instant**.
- Uses Hudi's incremental query type
- Instead of reconstructing the entire table, it retrieves the data affected by later commits.

Delete

- A DELETE operation creates a new commit on the Hudi timeline.
- The deleted record is no longer present in the **current table state**..
- Previous versions of the table can still contain the record.
- Therefore, time travel can show the record as it existed before the deletion

Rollback

- Rollback reverses a completed Hudi commit and restores the table to the state before that commit.
- Rollback operates on a specific commit.
- The effects of that commit are undone.

Compaction

In a Merge-on-Read table, updates can accumulate in delta log files instead of immediately rewriting the base files. Compaction merges these changes into new base files.

Indexing

- When Hudi receives an update or delete, it needs to locate the file containing the record identified by its record key.
- The Hudi index maps record keys to the locations of their records.

Metadata Table

- The Hudi Metadata Table stores metadata about the Hudi table so that Hudi can obtain information about the table without repeatedly scanning the underlying data files.
- It can maintain information such as: File listings, Record/file metadata, Column statistics etc.
- The data files contain the records; the index and metadata help Hudi efficiently locate and manage those records.

Hudi provides APIs that allow applications and data-processing engines to interact with Hudi tables and perform table-management operations.

In a spark environment,

- Hudi integrates directly with Apache Spark through its datasource and Spark SQL extensions.
- Spark SQL can be used to create, read, update, delete, and query Hudi tables.
- Hudi also provides table-management APIs, including REST-based interfaces, for interacting with Hudi table services without directly manipulating the underlying files.
- These APIs operate on the Hudi table abstraction, rather than requiring applications to manage Parquet files and Hudi metadata themselves.

1. What a "table" actually *is* in each system

Iceberg's mental model: a table is a sequence of snapshots. Every write produces a new **snapshot**. Iceberg doesn't inherently think in terms of "records", it thinks in terms of *which files make up the table at this version*. If you never set a primary key, Iceberg raises no issues;; a lot of Iceberg tables are pure append/overwrite with no concept of record identity at all.

Hudi's mental model: a table is a sequence of commits, and every row has an identity. Every table has a **primaryKey** (record identity is not optional,). Hudi maintains a **timeline** (the **.hoodie** folder) that logs every action (insert, upsert, delete, compaction, clustering, rollback) as a discrete event. Because Hudi always knows "this incoming row is an update to *that* existing row," it can do things Iceberg wasn't originally built for, like resolving *which version of a record wins* using a **preCombineField**.

2. How an update physically happens.

- **Iceberg, copy-on-write:** the file containing the changed row is rewritten entirely. Simple, but expensive for small, frequent changes.
- **Iceberg, merge-on-read (newer addition):** instead of rewriting the whole file, Iceberg writes a small **delete file** that says "ignore this row in that file," and the query engine merges it in at read time.
- **Hudi, copy-on-write:** same idea as Iceberg's COW — rewrite the base Parquet file immediately.
- **Hudi, merge-on-read:** changes go into a **row-based delta log** appended next to the base file; a background **compaction** process later merges the log into a new base file. This is Hudi's original, signature design

3. Concurrency and conflict resolution

- **Iceberg** uses **optimistic concurrency at the snapshot level**: two writers race to create the next snapshot; whoever commits first wins, the loser retries.
- **Hudi** resolves conflicts at the **record level**, using **preCombineField**. If two versions of the same primary key arrive, Hudi keeps the one with the greater value in that field (e.g. a later **updated_at**) regardless of write order.

4. . Time travel and incremental reads

- **Iceberg**: query by snapshot ID or timestamp — very natural since the whole table is snapshot-versioned.
- **Hudi**: query **as.of.instant** a commit, or run a genuinely **incremental** read (**hoodie.datasource.query.type=incremental**) that returns *only the rows that changed* since a given commit.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Apache Druid

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Content in this slide deck has been compiled from Apache Airflow official documentation and supplementary academic resources. Information may have been sourced from books, research papers, and internet references. Credit and rights remain with the original authors. These slides are intended for classroom presentation only.

- An open-source, column-oriented database purpose-built for fast, ad-hoc analytical (OLAP) queries on event-driven data
- Blends ideas from data warehouses, time-series databases, and search systems into one engine
- Optimized to answer “how many / how much / trending how” questions over huge, constantly arriving datasets in sub-second time
- Not a general-purpose OLTP database: no arbitrary multi-table joins, no row-level transactional updates

HOW DATA IS ACTUALLY STORED

Druid datasource

→ Time chunk (eg. 1 hour)

→ Segment (immutable, columnar)

→ Compressed columns +

indexes

TRADITIONAL APPROACHES

- Data warehouses: powerful SQL, but query latency is seconds-to-minutes, and freshness is batch-cycle limited
- OLTP databases: row-level and fast to write, but not built to scan/aggregate billions of rows
- Search engines: fast filtering, but weak at numeric aggregation and rollups

APACHE DRUID

- Sub-second aggregation queries, even across trillions of rows
- Native streaming ingestion: data is queryable within seconds of arrival
- Segments are time-partitioned and pruned per query, so only relevant data is scanned
- Optional roll-up compresses raw events into pre-aggregated rows at ingest time

BIG DATA

Architecture Overview

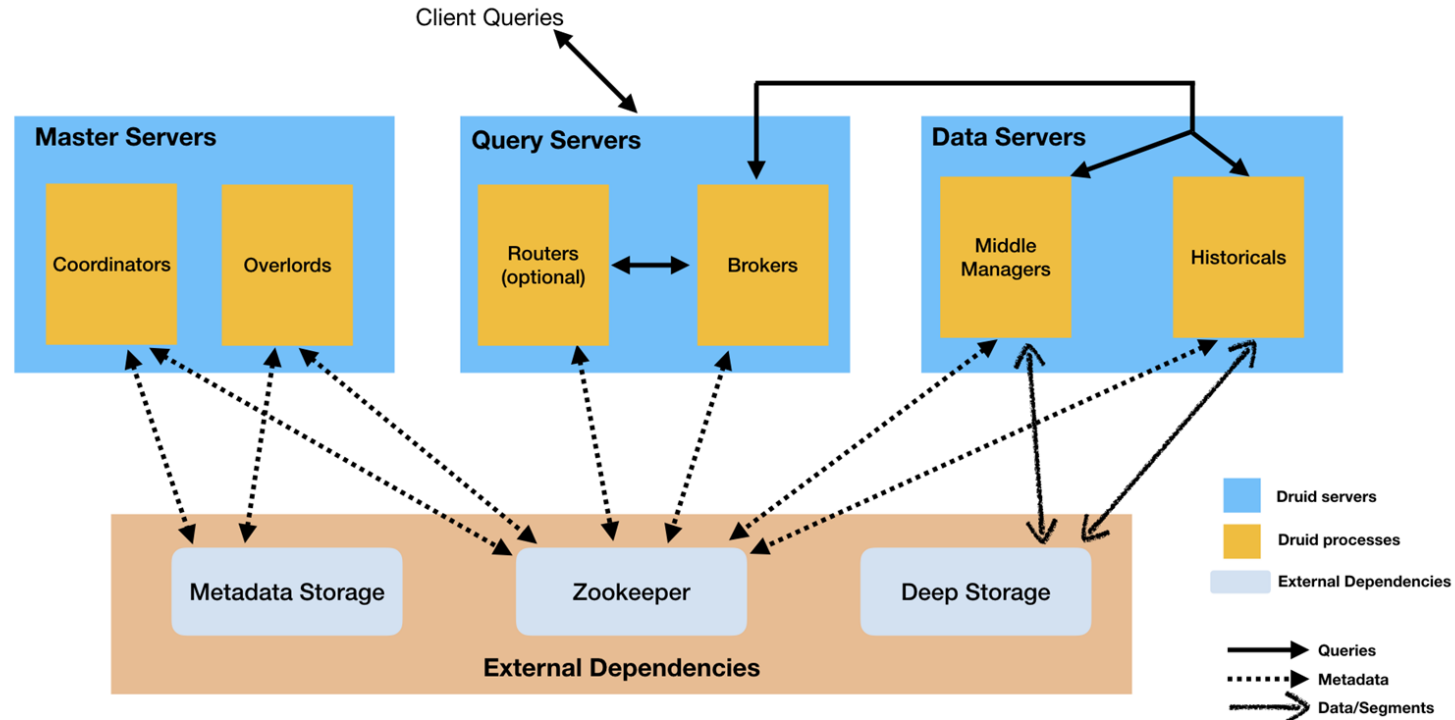
A Druid cluster is made of specialized processes, usually organized into three server types

Server Type	Processes	Responsibility
Master	Coordinator, Overlord	Manages data availability and assigns ingestion workloads
Query	Broker, Router	Handles and routes queries from external clients
Data	Historical, MiddleManager	Stores queryable segments and executes ingestion tasks

Query servers never touch storage directly,
Clients talk to the Broker (or Router), the Broker prunes and forwards to Data servers, and Data serves do the scanning. Master servers coordinate everything but never touch query traffic or raw data themselves.

BIG DATA

Architecture Overview



COORDINATOR

Manages data availability across the cluster. It assigns, balances, and drops segments on Historical.

OVERLORD

Controls assignment of ingestion workloads and coordinates segment publishing

BROKER

Receives external queries, prunes to relevant segments, forwards subqueries, and merges results

ROUTER

Optional unified API gateway in front of Brokers, Coordinates, and Overlords. It also hosts the web console.

HISTORICAL

Downloads immutable segments from deep storage and serves queries against them.

MiddleManager

Executes ingestion tasks and indexes streaming data in real time, via per-task Peon processes

THE HIERARCHY

- A datasource is Druid's logical equivalent of a table
- Each datasource is partitioned by time into time chunks (e.g. hourly, daily)
- Every time chunk is made of one or more segments : the fundamental unit of storage

WHAT A SEGMENT IS

- An immutable, columnar file: each column stored, compressed, and indexed separately
- Bitmap indexes on dimension columns make filtering extremely fast
- Once published, segments move to deep storage (S3, HDFS, ...) and are handed off to Historicals

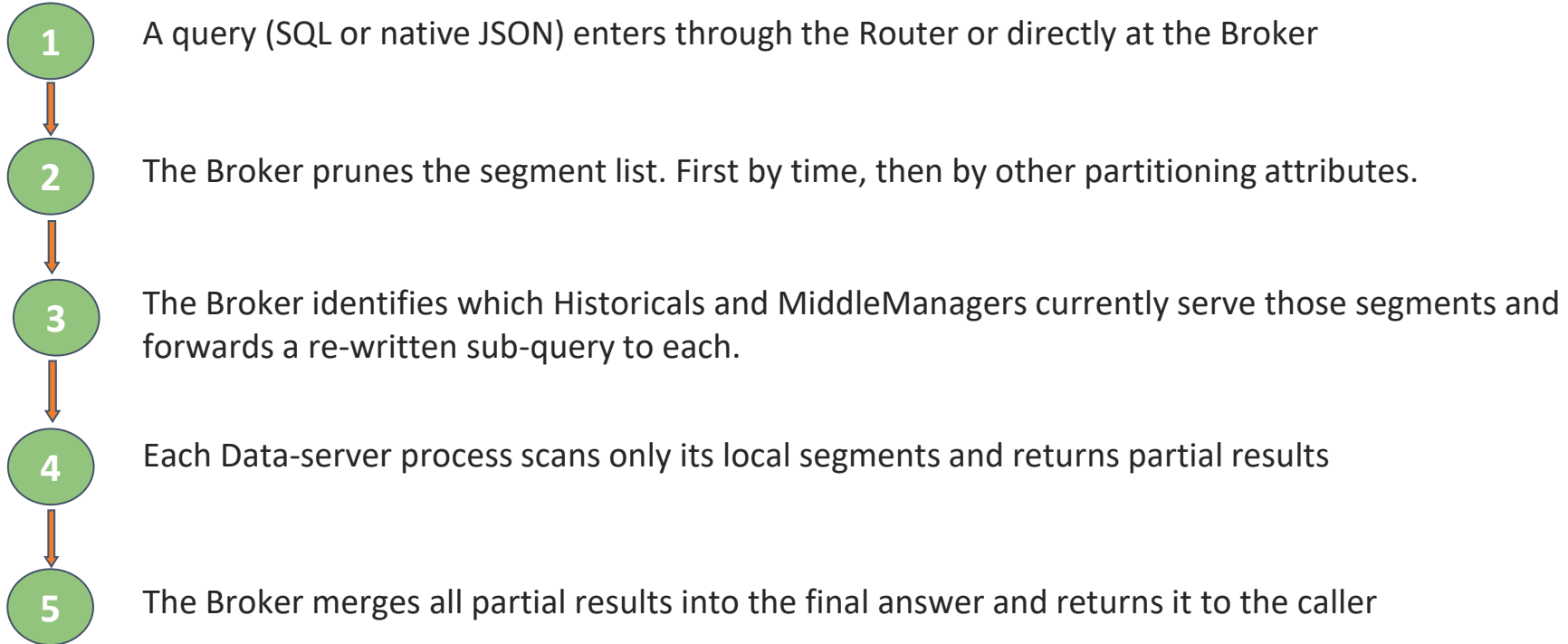
```
datasource: web_events
time_chunk  2026-08-14T09:00:00
├ segment_0000.smoosh
├ segment_0001.smoosh
└ segment_0002.smoosh
columns inside a segment:
timestamp   [compressed]
country      [dictionary + bitmap]
bytes_sent  [compressed metric]
```

STREAMING

- A supervisor manages MiddleManager tasks that read continuously from Kafka or streaming service.
- Data is queryable within seconds of arrival
- Exactly-once ingestion semantics
- Segments are handed off automatically as time chunks close

BATCH

- Native batch or Hadoop-based ingestion loads a bounded dataset in one job.
- Used for historical backfills and reprocessing
- Can reindex existing segments to change granularity or schema
- No continuous supervisor, the job runs once and completes





THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Big Data Algorithms – Relational Operations

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

- Relational algebra overview - Overview
- Select and Project with MapReduce
- Set Operations with MapReduce
- Join
- Grouping and Aggregation

Relational Operations



Overview of Relational Algebra

- The relational algebra is a procedural query language.
- It consists of a set of operations that take one or two relations as input and produce a new relation as their result.

The fundamental operations in the relational algebra are

- 1) Select
- 2) Project
- 3) Union, intersection, set difference
- 4) Cartesian product
- 5) Rename
- 6) Natural Join
- 7) Grouping
- 8) Aggregation

- Relations

- Relations are represented or expressed as **Tables**
- Tables have number of **Rows**. With each of the row having the same structure
- Rows (also called **tuples**) represents a singly implicitly structured data item
- Each piece of data is stored along the **row**.
- Each Row is broken up into **columns**. Each column represents an **attribute**
- *Relationship is defined across all columns as a relationship say $R(A_1, A_2, \dots, A_n)$*

- Relational Operators

- Selection: $\sigma_c(R)$ select from R according to condition C
- Projection $\pi_s(R)$ select from R subset of attributes S
- Union eg. $A \cup B$, intersection eg. $A \cap B$, difference eg. $(A - B)$
- Natural join $\bowtie$
- Grouping: partition R according to attributes G
- Aggregation: $SUM, COUNT, AVG, MAX, MIN$

id	Name	Role	Team
1	Virat Kohli	Captain	RCB
2	Gautham Gambhir	Captain	KKR
3	Anil Kumble	Coach	MI
4	Virender Sehwag	Coach	KXIP

1. Write an SQL query to list
 - a) All the details for the Coaches.
 - b) Only the names of the coaches
 - c) Total #coaches.
2. What type of a relational operation are we using?

BIG DATA

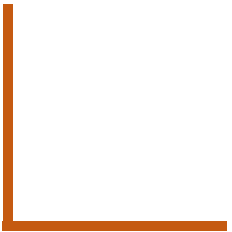
Simple Problem now from a bigdata perspective

- Data in a database is well structured.
- Instead assume that data is stored in a csv file in HDFS as shown below.

```
1, Virat Kohli, Captain, RCB  
2, Gautham Gambhir, Captain, KKR  
3, Anil Kumble, Coach, MI  
4, Virender Sehwag, Coach, KXIP
```

- Now we need to perform the query.

Select and Project in MapReduce



How will the map output look for the query 1a ?

- **map**

- Read each row t of table
- Check if it satisfies condition C
- If so, output (t, t)

Map

<"3, Anil Kumble, Coach, MI","3, Anil Kumble, Coach, MI">

<"4, Virender Sehwag, Coach,KXIP","4, Virender Sehwag, Coach,KXIP">

- **Reduce**

- do nothing

Reduce

3, Anil Kumble, Coach, MI

4, Virender Sehwag, Coach,KXIP

- **map**

- Read each row t of table
- Calculate subset of attributes t'
- Output (t', t')

Map

<"Anil Kumble", "Anil Kumble">
<"Virender Sehwag", "Virender Sehwag">

- **reduce**

- Eliminate duplicates
 - $(t', [t', t', t']) \rightarrow (t', t')$

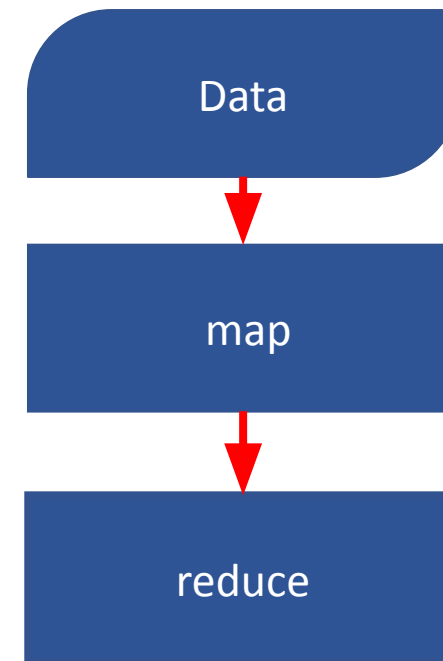
Reduce

Anil Kumble
Virender Sehwag

Relational Operations requiring two input files – Set operations

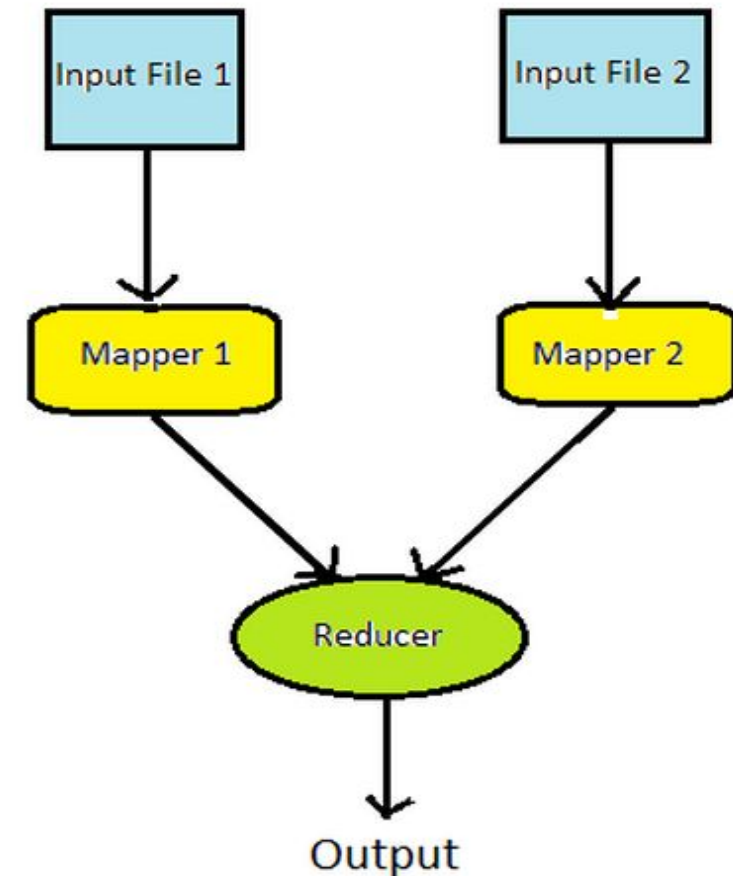


- Union $R \cup S$
 - R and S have the same structure (Schema)
 - Output records in R or S
 - *We need to solve a problem here*
- The basic problem :
 - Need to read in 2 input files
 - produce 1 output file
- Reading multiple input files
 - Same problem in Matrix-vector multiplication
 - Need to read in matrix $\mathbf{M}$ and vector $\mathbf{v}$



- map reads in one file
- If we have multiple input files, we can try to combine them into one file
 - This can be used as input to map
- Problem: multiple input files can be combined in many ways
 - We can have all the records of one file followed by all the records of another file
 - Or merge the two files and sort the records
 - Files may not have the same structure

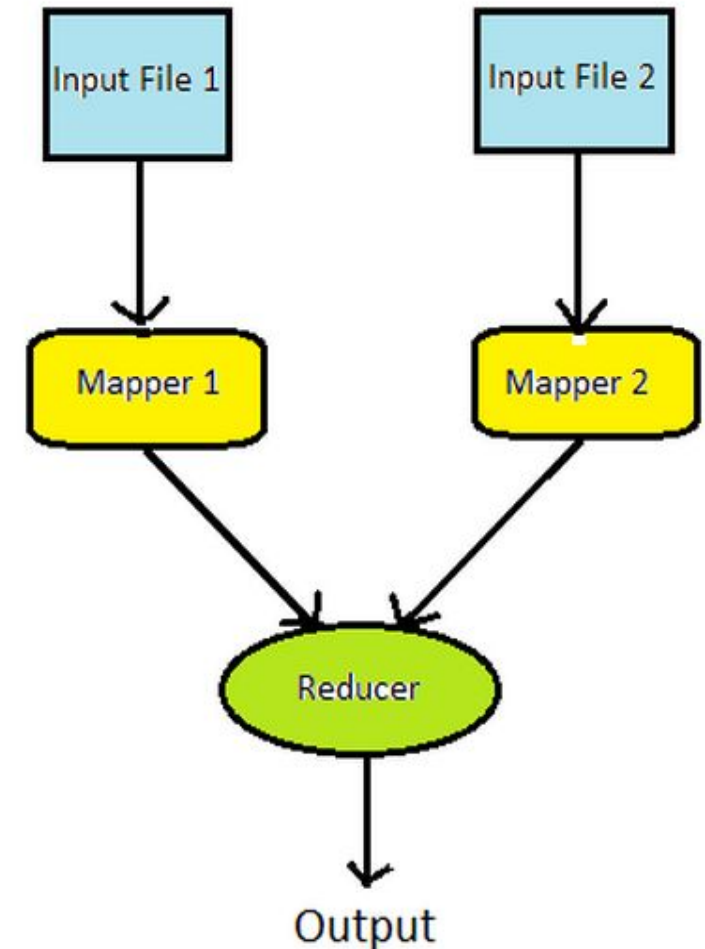
- Use map reduce itself to merge the files
- Example
 - We have two mappers
Each mapper reads in one file
 - Each mapper writes a key-value pair
 - The reducer uses the keys to merge the files
 - This output can be used as input to subsequent map reduce passes



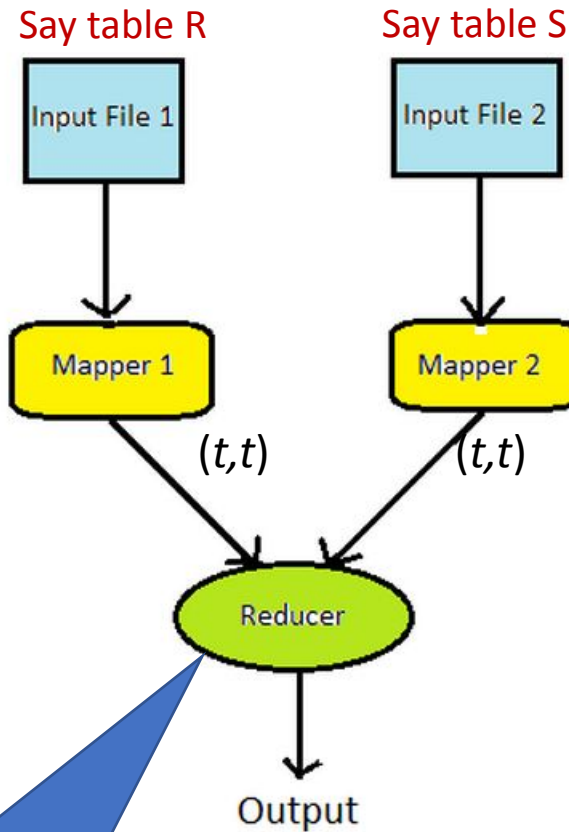
Union *R U S*

- Need to read in 2 input files and produce 1 output file
- Need to have 2 mappers reading different input files
- Can be done using *MultipleInput* option in Hadoop as shown below for both the mapper classes

```
MultipleInputs.addInputPath(job,new Path(args[0]),  
                             TextInputFormat.class, Mapper1.class);  
  
MultipleInputs.addInputPath(job,new Path(args[1]),  
                             TextInputFormat.class, Mapper2.class);
```



- mapper 1
 - Read each row of table R
 - Output (t,t)
- mapper 2
 - Read each row of table S
 - Output (t,t)
- reducer
 - Eliminate duplicates if any
 - $(t,[t,t]) \rightarrow (t,t)$



Should remove the
duplicates

- Given the following input for two files

- File 1

- A
- B
- C
- D

- File 2

- A
- E
- F
- C

- Show (for the Union algorithm)
 - Input and output of mapper 1
 - Input and output of mapper 2
 - Input and output of reducer

- Mapper 1 output

- A , A
- B, B
- C, C
- D, D

- Mapper 2 output

- A, A
- E, E
- F, F
- C, C

- Reducer Input

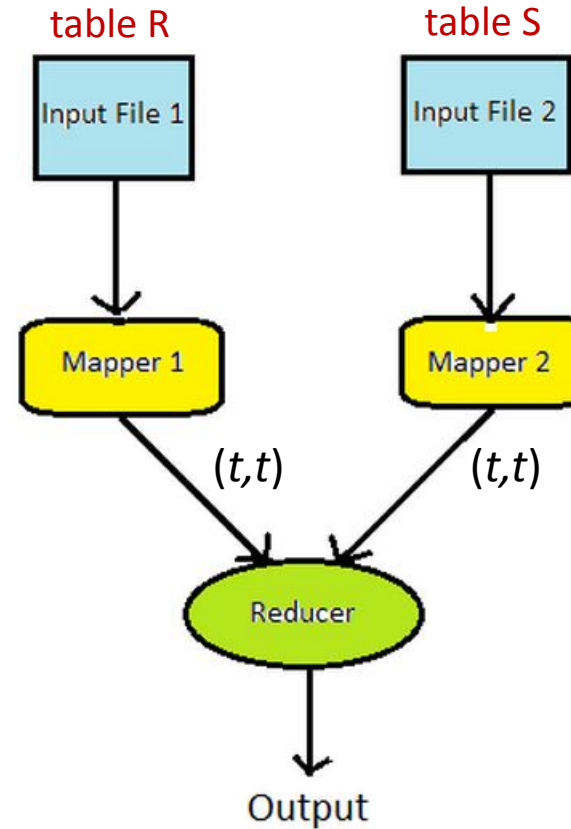
- A, [A, A]
- B, B
- C, [C, C]
- D, D
- E, E
- F, F

Value A has got one value A from file1 mapper and another value A from file2 mapper

- Reducer Output

- A
- B
- C
- D
- E
- F

- Compute $R \cap S$
- mapper 1
 - Read each row of table R
 - Output (t,t)
- mapper 2
 - Read each row of table S
 - Output (t,t)
- reducer
 - Output only duplicates
 - $(t,[t,t]) \rightarrow (t,t)$



Just choose the duplicates from mapper 1 and mapper 2 outputs

- Compute $R - S$
 - All rows in R , not in S

- mapper 1

- Read each row of table R
- Output (t, R)

rows, identification
that its table is R

- mapper 2

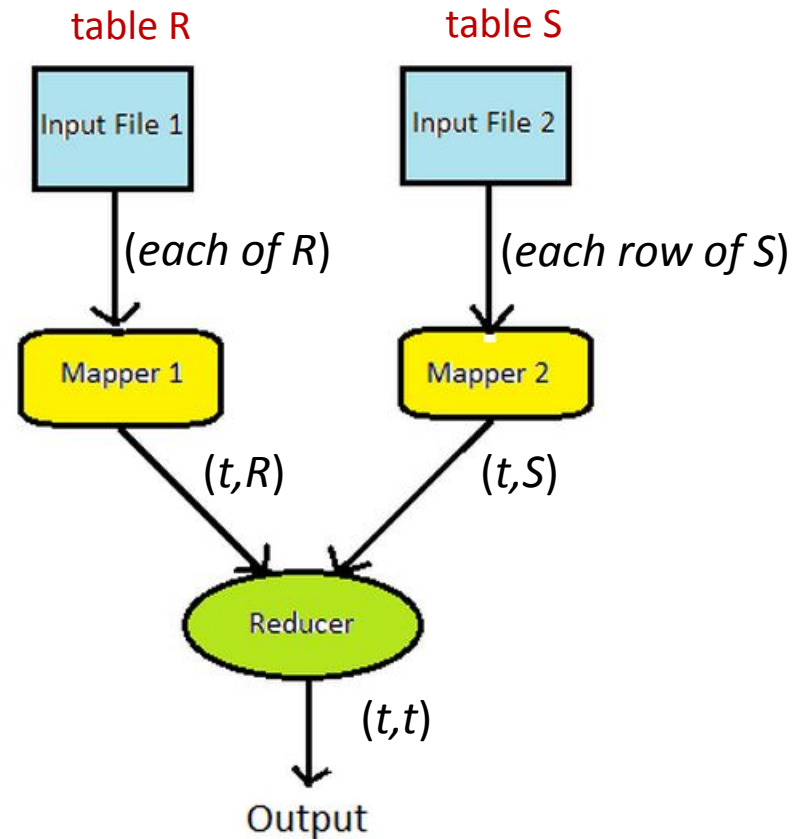
- Read each row of table S
- Output (t, S)

rows, identification
that its table is S

- reducer

- Output only
 - $(t, [R]) \rightarrow (t, t)$

rows, from only
table R



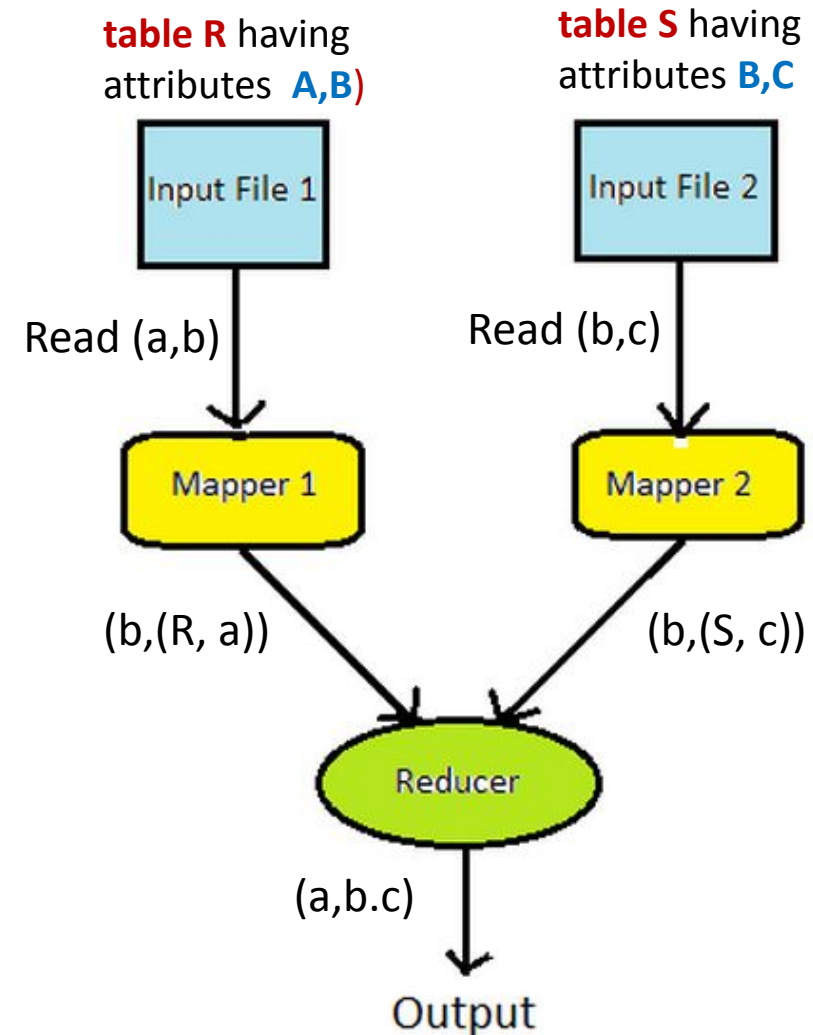
Join



BIG DATA

Natural Join

- Join R and S on attributes B
 - A, C are the other attributes in R,S
- mapper 1
 - Read (a,b) of R, output (b,(R,a))
- mapper 2
 - Read (b,c) of S, output (b,(S,c))
- reducer
 - for each pair b,(R,a)) and (b,(S,c)), output (a,b,c)



- Given the following input for two files
 - Table Employee E(Name, age)
 - Gabbar 35
 - Viru 37
 - Jai 33
 - Baldev 44
 - Basanti 31
 - Table Dept D(Name, Dept)
 - Gabbar Bandit
 - Viru Hero
 - Jai Hero
 - Baldev Police
 - Basanti Heroine

- Show (for the Natural Join algorithm)
 - Input and output of mapper 1
 - Input and output of mapper 2
 - Input and output of reducer

- **Mapper 1 Output**

- Gabbar, (E, 35)
- Viru, (E, 37)
- Jai, (E, 33)
- Baldev, (E, 44)
- Basanti, (E, 31)

- **Mapper 2 Output**

- Gabbar, (D, Bandit)
- Viru, (D, Hero)
- Jai, (D, Hero)
- Baldev (D, Police)
- Basanti (D, Heroine)

- **Reducer Input**

- Gabbar, (E, 35), (D, Bandit)
- Viru, (E, 37), (D, Hero)
- Jai, (E, 33), (D, Hero)
- Baldev, (E, 44), (D, Police)
- Basanti, (E, 31), (D, Heroine)

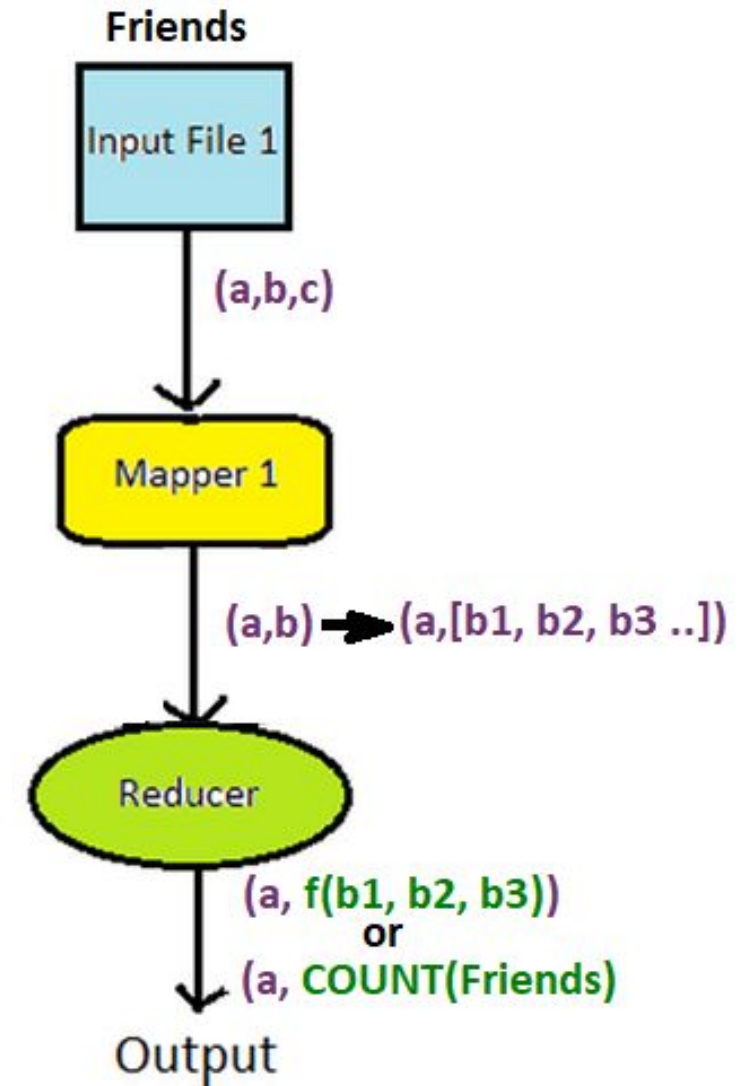
- **Reducer Output**

- Gabbar, 35, Bandit
- Viru, 37, Hero
- Jai, 33, Hero
- Baldev, 44, Police
- Basanti, 31, Heroine

Grouping and Aggregation



- For relation ***R*** (***A,B,C***) group by ***A*** and aggregate by function ***f(B)***
 - Social networking site has a relationship ***Friends*** (***User, Friend, Date of friendship***)
 - Grouping by ***User*** and aggregating by ***COUNT (Friend)*** produces a table
 - First column is the ***User***
 - Second column is the **count of *Friends* for the *User***



- **map**

- for each line **(a,b,c)** (e.g., **User, Friend, Date**)
- Output **(a,b)** (e.g., **User, Friend**)

- **reduce**

- Aggregate **(a, [b1 , b2 , b3 , ...])** into **(a, f(b1 , b2 , b3 , ...))**

- Given the following input file
 - Table Dept D(Name, Dept)
 - Gabbar Bandit
 - Viru Hero
 - Jai Hero
 - Baldev Police
 - Basanti Heroine
- Determine the #employees in each department
 - Show map Output
 - Show reducer Input/output

- **Mapper**

- **Bandit 1**
- **Hero 1**
- **Hero 1**
- **Police 1**
- **Heroine 1**

- **Reducer Input**

- **Bandit 1**
- **Hero [1,1]**
- **Police 1**
- **Heroine [1]**

- **Reducer Output**

- **Bandit 1**
- **Hero 2**
- **Police 1**
- **Heroine 1**



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Big Data Algorithm Complexity

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

- Motivation – Algorithm complexity
- Communication Cost Complexity Model
- 3 Way joins with the communication cost complexity
- Key parameters
- Similarity join - analysis

Motivation – Algorithm Complexity

- So far, we have looked at MapReduce algorithms
- However, for a particular problem, there could be many algorithms
- Which algorithm should we choose?
- This is why we study complexity of MapReduce
- We will actually study complexity of workflow systems
 - Generalization of MapReduce
 - Many important Big Data systems are workflow systems

- Consider the following two problems
 - Matrix multiplication
 - Database query
- What would be the complexity of these algorithms when executing on a single node?
- What does complexity depend on?



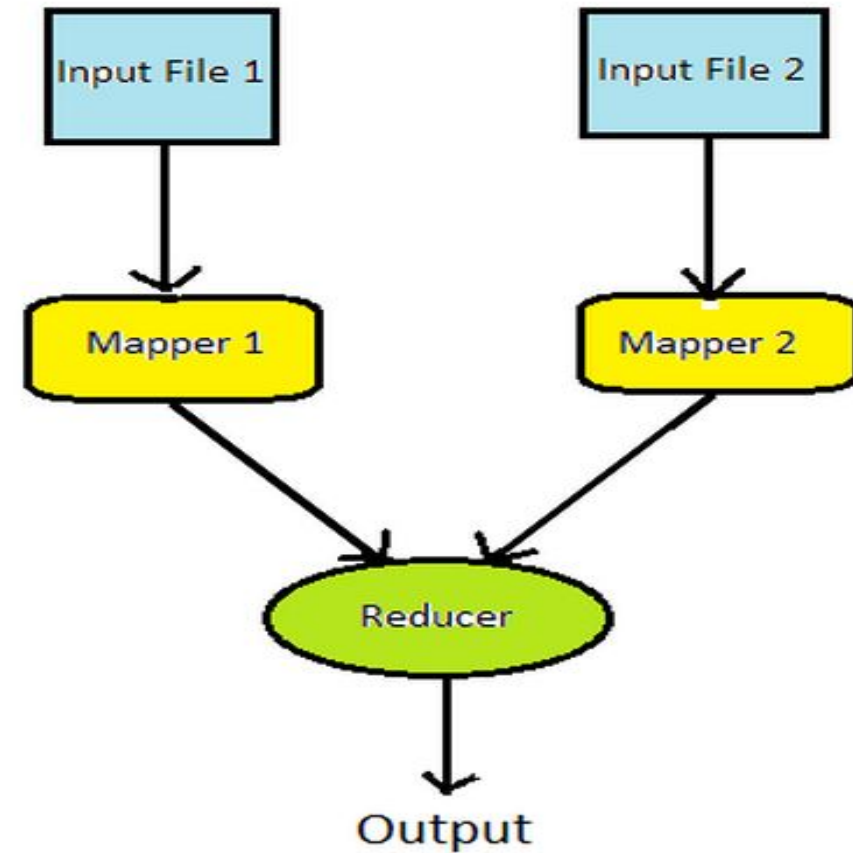
- Matrix multiplication
 - Expressed in terms of the bound on total #computations performed
- Database query
 - Complexity depends on disk read



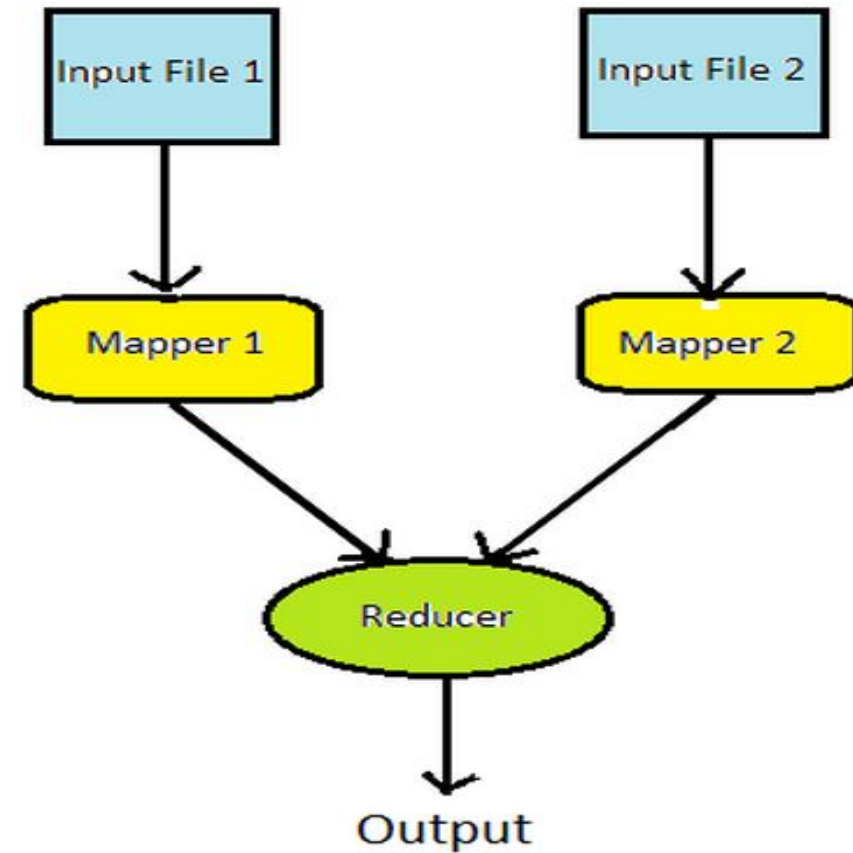
- Communication cost: size of input
- Why communication cost?
 - Algorithm tends to be linear in data
 - Network speed $\ll$ CPU speed
 - Disk speed $\ll$ CPU speed
 - Major time could be communication time

- Why only input size?
 - Output is input to some other task
 - Final output is generally small by aggregation
 - Otherwise not human-readable

- Mapper input complexity = $r+s$
 - Read data from disk
- Reducer input complexity = $r+s$
 - Network reads
- Total Complexity: $O(r+s)$

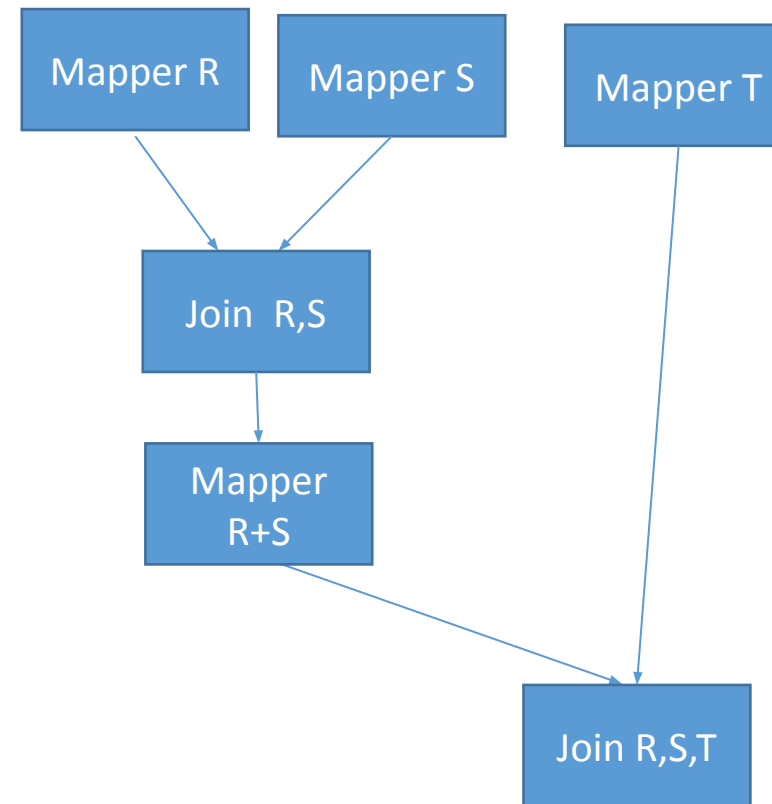


- Consider three relations R, S and T
- How will you perform a join using map reduce across the three?
- Estimate the complexity

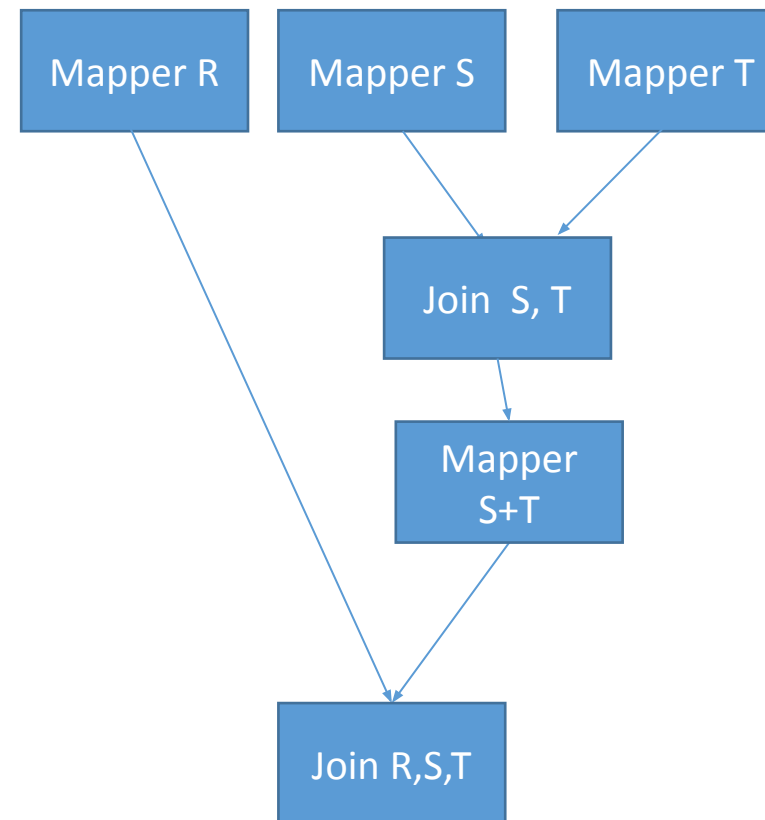


3 way join complexity

- 2 MapReduce phases
- Case 1: Join R, S and then join T
 - Input to Mapper 1: r
 - Input to Mapper 2: s
 - Input to Reducer 1: $r+s$
 - Let p be the probability of match between r, s
 - Input to Mapper 3: prs
 - Input to Mapper 4: t
 - Input to Reducer 2: $t+prs$
- Total Complexity: $O(r+s+t+prs)$



- Case 1: Join S, T and then join R
 - Input to Mapper 1: s
 - Input to Mapper 2: t
 - Input to Reducer 1: $s+t$
 - Let q be the probability of match between s, t
 - Input to Mapper 3: qst
 - Input to Mapper 4: r
 - Input to Reducer 2: $r+qst$
- Total Complexity: $O(r+s+t+qst)$
 - Depends upon join order
 - If $p \sim q$, first join could be whichever of rs, st, rt is the smallest



Key performance parameters

- Can make communication cost very low by executing all tasks on single CPU
- However, program may run slowly
- Need to consider *wall clock time*
 - Time taken for entire job to finish
- Dividing jobs could increase communication but reduce wall clock time
 - Need to trade off communication time, wall clock time

- Reducer size q
 - Max # of values that can have the same key
 - Not the number of reducers
 - If q is small, there can be more reducers
 - Suppose the number of Map outputs is T
 - Max number of reducers = T/q
 - This will reduce the wall clock time
 - But increase the communication cost
- Replication rate r
 - $r = (\text{\#key value pairs output by Mapper}) / (\text{\# input records to Mapper})$
 - Average communication cost from Map tasks to Reduce tasks

2.6.1 Reducer Size and Replication Rate

Let us now introduce the two parameters that characterize families of MapReduce algorithms. The first is the *reducer size*, which we denote by q . This parameter is the upper bound on the number of values that are allowed to appear in the list associated with a single key. Reducer size can be selected with at least two goals in mind.

1. By making the reducer size small, we can force there to be many reducers, i.e., many different keys according to which the problem input is divided by the Map tasks. If we also create many Reduce tasks – even one for each reducer – then there will be a high degree of parallelism, and we can look forward to a low wall-clock time.
2. We can choose a reducer size sufficiently small that we are certain the computation associated with a single reducer can be executed entirely in the main memory of the compute node where its Reduce task is located. Regardless of the computation done by the reducers, the running time will be greatly reduced if we can avoid having to move data repeatedly between main memory and disk.

The second parameter is the *replication rate*, denoted r . We define r to be the number of key-value pairs produced by all the Map tasks on all the inputs, divided by the number of inputs. That is, the replication rate is the average communication from Map tasks to Reduce tasks (measured by counting key-value pairs) per input.

Similarity Joins

- Assume we have a database of 1 million images
- Each image is 1 MB
- Total DB size = 1 TB
- Assume there is a function $s(x,y)$ which determines how similar two images x,y are
 - $s(x,y) = s(y,x)$
- Problem: output all pairs x,y such that $s(x,y) > t$

- Assume each image P_i has an index i
- Mapper
 - Reads in (i, P_i)
 - Generates all pairs $(\{i, j\}, \{P_i, P_j\})$
- Reducer
 - Reads $(\{i, j\}, \{P_i, P_j\})$
 - Computes $s(P_i, P_j)$

- What is the
 - Communication cost of the naïve algorithm?
 - Parallelism of the naïve algorithm?
 - What is the replication rate for the algorithm?



- What is the
 - Communication cost of the naïve algorithm?
 - Parallelism of the naïve algorithm?
- Algorithm doesn't work
 - Data to be transmitted = $1,000,000 \times 999,999 \times 1,000,000$ bytes = 10^{18}
 - Communication cost is $\sim n^2$ where n is the number of images (extremely high)
 - However, potential parallelism is very high
 - Reducer size is **very LOW** ~~high~~ ☐ since no list has more than two values, or a total of 2MB input.
 - since 10^{18} key value pairs are generated, each can be processed in parallel by a different reducer. So **parallelism is high**
 - Replication rate is **very high** -> each piece of data copied 999,999 times, so huge amount of data sent from mapper to reducer.

- Do everything on one node
- Mapper
 - Reads in (i, P_i)
 - Generates all pairs $(\{i, j\}, \{P_i, P_j\})$
- Reducer (runs on same node as mapper)
 - Reads $(\{i, j\}, \{P_i, P_j\})$
 - Computes $s(P_i, P_j)$
- No communication cost
- Very low parallelism (wall clock time high)

Send one pair to each reducer

- High communication cost (bad)
- High parallelism (good)

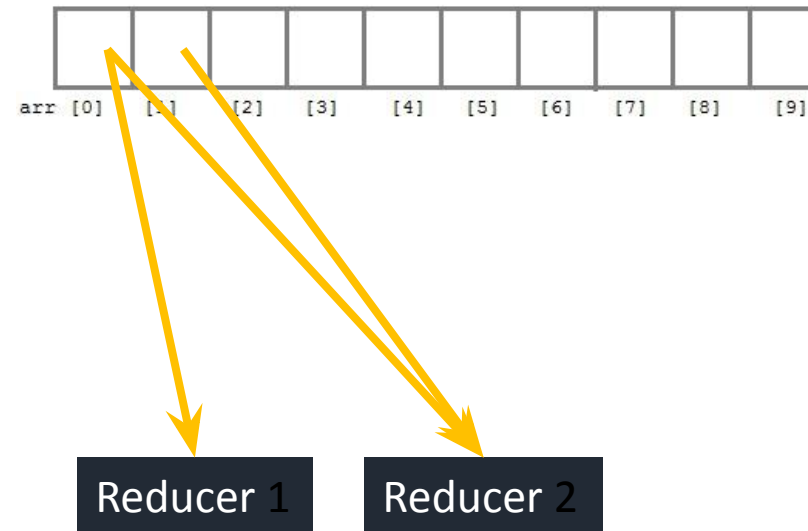
Do everything on one node

- Low parallelism (bad)
- Low communication cost (good)

Can we get something in between?

- Overview

- Group images
- Read in two groups of images in a single reducer
- Store them in memory
- Compare all pairs from those groups
- Output results



- Suppose the groups are $G_0, \dots, G_{99}$
- Group G_0 is sent to nodes 0, 1, ..., 98
 - Why?
- Group G_0 has to be compared with 99 other groups
- Group G_1 is sent to?
 - ~~0, 1, ..., 98~~
- Group G_1 is sent to 0, 99, 100, ...196 (0+98 other nodes)
- Group G_2 is sent to?
- Group G_2 is sent to 1, 99, 197, 198, ... 293 (1, 99 +97 other nodes)
- Group G_3 is sent to 2, 100, 197, 294, ...389 (2, 100, 197, +96 other nodes)

- If there are g groups
- The number of images per group $m=n/g$
- Each group is sent to $g-1$ servers
- Total number of messages = $g(g-1)$
- **Total data = $mg(g-1) = n(g-1) \sim ng$**
- **Parallelism = no of nodes = $(g-1) + (g-2) + (g-3) + \dots = g(g-1)/2 = O(g^2)$**
- Another way = no of nodes = ${}^nC_2 = g(g-1)/2$

- Suppose we have groups of 100
 - How many groups are there?
 - How many nodes is each group sent to?
- What is the
 - Communication cost of the algorithm?
 - Parallelism of the algorithm?

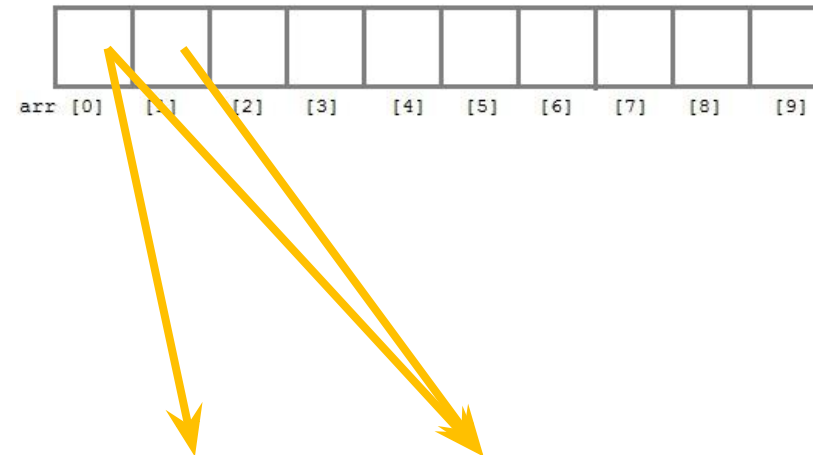


- Naïve: approximately 1,000,000x1,000,000 images = 10^{12} images, parallelism = 10^{12}
- Example: group images into groups of 100
 - Communication cost: 1,000,000 images x 100 groups = 10^8 images
 - Parallelism = 10^4 (100 groups compared with 99 other groups $\sim 10^4$)
- Example: group images into groups of 1000
 - Communication cost = 1,000,000 x 1000 = 10^9 images
 - Parallelism = 10^6
- Trade-off: increasing group size
 - Increases communication complexity (more reads)
 - Reduces wall clock time (increases parallelism)

- On next slide
- Note
 - Group-based algorithm is discussed in book as if mapper sends pictures to reducer
 - In previous slides, we have discussed as if reducer reads in pictures from disk
 - From communication cost complexity both are the same
 - Performance: probably better to read from disk

- Overview

- Group images
- Read in two groups of images in a single reducer
- Store them in memory
- Compare all pairs from those groups
- Output results



- Complexity

- Communication cost $\sim ng$ where g is the number of groups
- Can still get good parallelism

- Group images into g groups
- Mapper
 - Input: (i, P_i)
 - Find u = group to which image i belongs
 - Output $g-1$ key-value pairs $(\{u, v\}, i, P_i)$ for all $v \neq u$
- Reducer:
 - There is one reducer for each unique key $\{u, v\}$
 - Use the input to store images for groups u, v
 - Compare all pairs of images in groups u, v
 - If $v=u+1$, then compare all pairs of images in group u

- If group size is 1000
- Need ~2GB to store 2000 images in memory
- Need ~500,000 reducers
- If we have a 10,000 node cluster
 - Can do computation in 50 passes
 - Speedup of 10,000

- There could be many MapReduce algorithms to implement a particular functionality
 - Matrix multiplication
 - Multi-way joins
- There are two factors to consider when selecting an algorithm
 - Communication complexity: volume of data that is input to the different phases of the algorithm
 - Wall clock time: Total elapsed time for algorithm
 - Depends upon parallelism
 - Frequently there is a trade-off between these factors
 - Group size

- Mining of Massive Datasets
 - Rajaraman et. Al.
 - Chapter 2.4-2.5

2.3.11	Exercises for Section 2.3	40
2.4	Extensions to MapReduce	41
2.4.1	Workflow Systems	41
2.4.2	Recursive Extensions to MapReduce	42
2.4.3	Pregel	45
2.4.4	Exercises for Section 2.4	46
2.5	The Communication Cost Model	46
2.5.1	Communication-Cost for Task Networks	47
2.5.2	Wall-Clock Time	49
2.5.3	Multiway Joins	49
2.5.4	Exercises for Section 2.5	52
2.6	Complexity Theory for MapReduce	54
2.6.1	Reducer Size and Replication Rate	54
2.6.2	An Example: Similarity Joins	55
2.6.3	A Graph Model for MapReduce Problems	57
2.6.4	Mapping Schemas	58
2.6.5	When Not All Inputs Are Present	60
2.6.6	Lower Bounds on Replication Rate	61
2.6.7	Case Study: Matrix Multiplication	62
2.6.8	Exercises for Section 2.6	66
2.7	Summary of Chapter 2	67
2.8	References for Chapter 2	69
3	Finding Similar Items	73
3.1	Applications of Near-Neighbor Search	73
3.1.1	Jaccard Similarity of Sets	74
3.1.2	Similarity of Documents	74
3.1.3	Collaborative Filtering as a Similar-Sets Problem	75



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Columnar Databases : HBase

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Chapter 20 – T1

- **Hbase**

- Distributed column oriented database built on top of HDFS
- Data is logically organized as rows/columns of a table

- **Cassandra**

- Distributed database – peer to peer built by facebook
- Inspired by Dynamo DB
- Same data model as Hbase – inspired by BigTable

2. Column Families

HBase schema consists of several **Tables**

Each table consists of a set of **Column Families**

Columns are not part of the schema

HBase has **Dynamic Columns**

Because column names are encoded inside the cells different cells can have different columns

“School” column family has different columns in different cells



Name

HarryPotter

Voldemort

Data

Info:{height:”4.5ft”, age: “11@2011”}

School:{House:”Gryffindor”,

Sports:”Quidditch”}

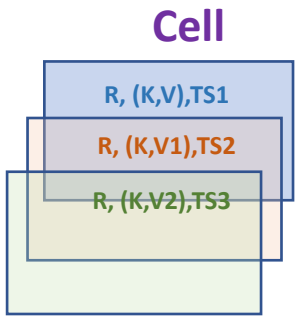
Info:{height:”7ft”, age: “50”}

School:{House:”Syltherin”, Role:”Prefect”}

- HBase Data Model is designed to handle semi-structured data that may differ in field size, and would be in the form of data and columns.
- The data model’s layout partitions the data into simpler components and spread them across the cluster.
- HBase’s Data Model consists of various logical components
 - **Table**: Data represented as a collection of rows sorted on RowID
 - **Row**: Collection of column families identified by **RowID** (Row Key), a byte array, serving as the primary key for the table and is indexed for fast lookup
 - **Column**: Collection of key-value pairs – represented by ColumnFamilyName:ColumnName
 - **Column family**: Collection of variable number columns
 - **Cell**: Stores data and is a combination of {row key, column, timestamp/version} tuple as a byte array
 - Timestamp (System timestamp) or any other unique version number within a RowId, for the cell

Table

Rowid	Column Family			Column Family			Column Family			Column Family		
	col1	col2	col3	col1	col2	col3	col1	col2	col3	col1	col2	col3
1												
2												
3												

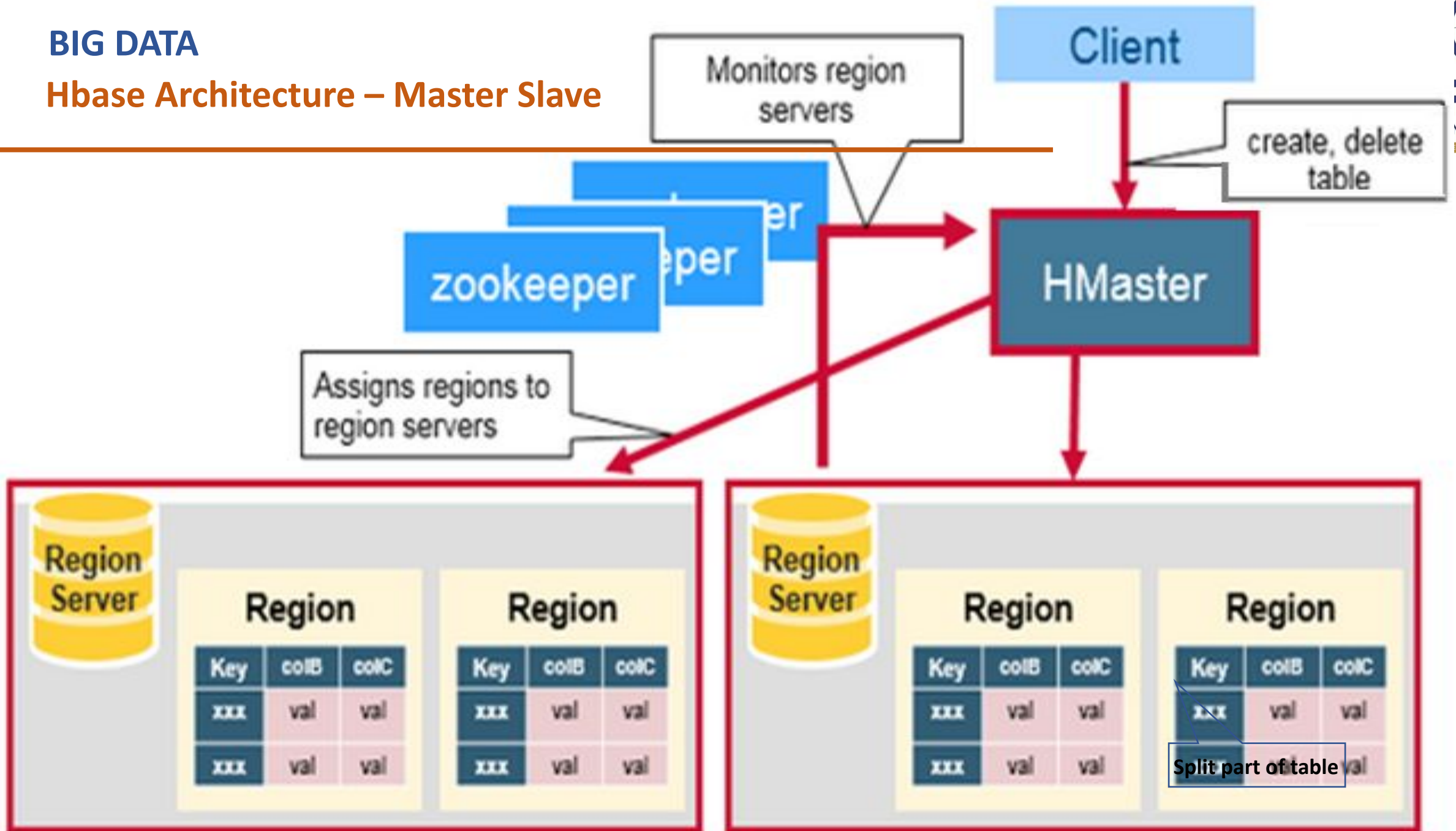


Hbase Architecture



BIG DATA

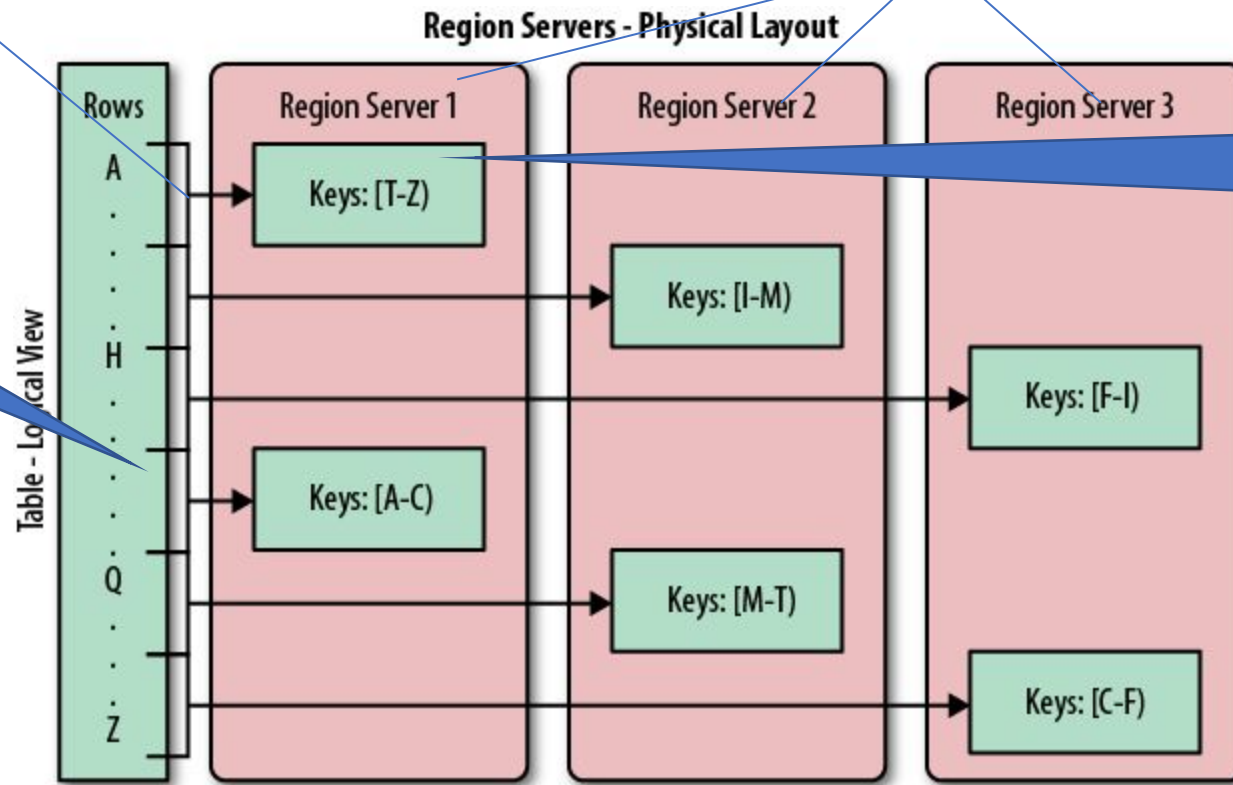
Hbase Architecture – Master Slave



Tables horizontally partitioned into regions

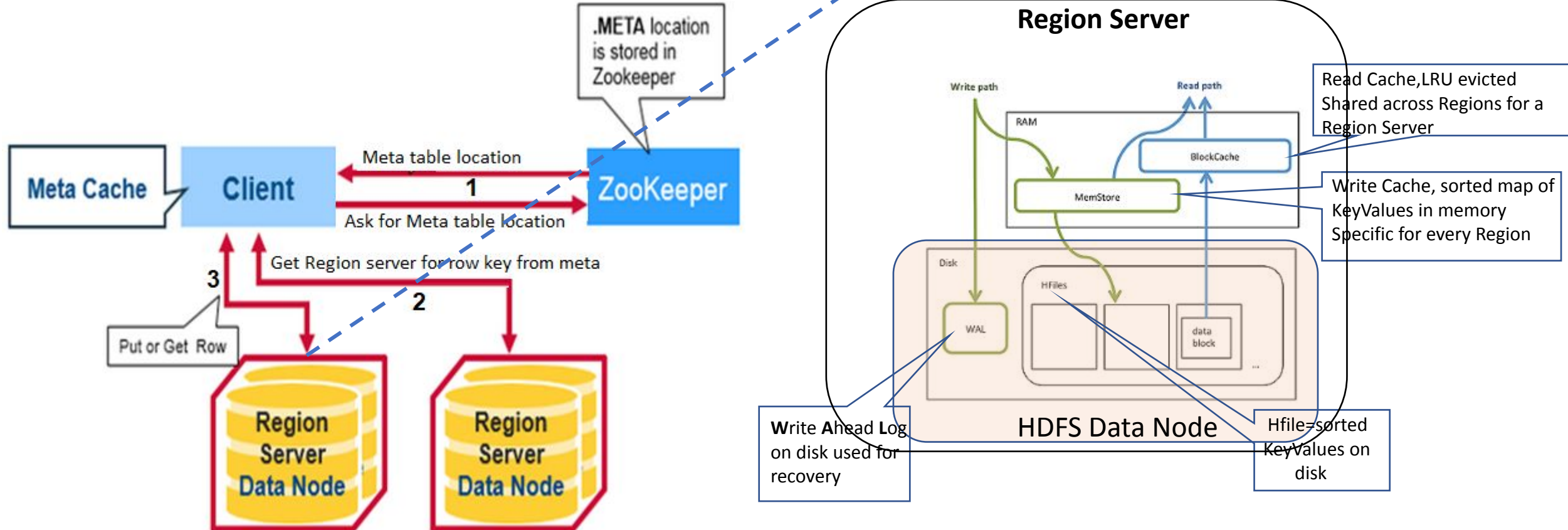
Regions stored on region servers

META Table:
keeps track of
regions



Start with a single region and
then master monitors load
and splits into multiple
regions dynamically

- BlockCache and MemStore reside in random-access memory (RAM).
- HFiles and the Write Ahead Log are persisted to HDFS.
- Simplified Read/Write paths would be



Hbase usage



```
hbase(main):001:0> create 'test', 'data'  
0 row(s) in 0.9810 seconds
```

Column
family name

Table name

```
hbase(main):003:0> put 'test', 'row1', 'data:1', 'value1'  
hbase(main):004:0> put 'test', 'row2', 'data:2', 'value2'  
hbase(main):005:0> put 'test', 'row3', 'data:3', 'value3'
```

Row key

Column
name

Value

Getting a
specific row

```
hbase(main):006:0> get 'test', 'row1'  
COLUMN          CELL  
  data:1         timestamp=1414927084811, value=value1  
1 row(s) in 0.0240 seconds
```

```
hbase(main):007:0> scan 'test'  
ROW          COLUMN+CELL  
  row1       column=data:1, timestamp=1414927084811, value=value1
```

All rows



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Columnar Databases for Analytics

Prafullata Kiran Auradkar

Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Chapter 5 – R3 (Reference Book 3: “Big Data Analytics”, Rajkamal, Preeti Saxena, 1st Edition, McGraw Hill Education, 2019.)

Hadoop Available Storage Types



HDFS is good for batch processing (scans over big files)

Limitations

- Not good for record lookup
- Not good for incremental addition of small batches
- Not good for updates

- Doesn't store data
 - Uses HDFS or Hbase as actual store
- Provides an SQL Interface for querying data
- Data must be Structured: definite schema

Limitations

- Not good for record lookup
- Not good for incremental addition of small batches
- Not good for updates
- Not good for unstructured/semi-structured data

- Built on the Bigtable data model
- different in architecture

Advantages

- Fast record lookup
- Record level insertion
- Support for updates (Hbase creates new versions)
- Support for unstructured/semi-structured data

- **HDFS**
 - Unstructured data
 - Writes: no updates, only appends
 - Read entire file and analyse
- **Hive**
 - Structured data
 - Analytics via SQL
- **HBase/Cassandra**
 - Unstructured data
 - Arbitrary writes
 - Analytics

Which of these could be stored in HDFS, Hive or Hbase?

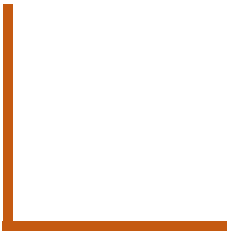
1. Parsed transaction logs of user activity in a website where relevant fields from the log have been extracted
2. Unparsed transaction logs of user activity
3. Database of users and friends at a social website, which is periodically analyzed for social networking analysis



Which of these could be stored in HDFS, Hive or Hbase?

1. Parsed transaction logs of user activity in a website where relevant fields from the log have been extracted
(HIVE)
2. Unparsed transaction logs of user activity **(HDFS)**
3. Database of users and friends at a social website, which is periodically analyzed for social networking analysis **(Hbase)**

Columnar Storage



- ◆ Cassandra is one of the popular column-family databases
- ◆ there are others, such as HBase, Hypertable, and Amazon DynamoDB

Feature	Relational Database (RDBMS)	Columnar Database
Data Storage	Row-based	Column-based
Primary Use Case	OLTP (Transactional Systems)	OLAP (Analytical Systems)
Query Efficiency	Efficient for small, row-level queries	Efficient for large, column-level queries
Data Integrity	Strong ACID compliance	Limited ACID compliance
Compression	Limited	High (column redundancy exploited)
Write Operations	Optimized for frequent writes	Optimized for bulk writes
Examples	MySQL, PostgreSQL, Oracle DB	BigQuery, Amazon Redshift, ClickHouse

Row Key	Info:height	Info:age	School:House	School:sports
HarryPotter	4.5ft	11	Gryffindor	Quidditch
Voldemort	7ft	50	Slytherin	

- **Row storage**: DB is stored as a single file, one row per line
- **Column storage**: each column is a separate file, one value per line

For using data, we need to perform an I/O to load data from disk. Which method does less I/O for

- Analysing the relationship between age and earnings
 - **Column storage**
- Adding a new row or read a row
 - **Row storage**

- **The first use of Databases were for transactions**
 - Read a person's bank balance
 - Update bank balance
 - Row storage used since more efficient for transactions
- **Column dbs**
 - Became popular with Big Data systems
 - More efficient for analytics, particularly if db is large
 - To handle unstructured data

STRUCTURED VS. UNSTRUCTURED DATA

Structured Data

High Degree of organization, such as a relational database

Column	Value
Patient	Joe Brown
Date of Birth	02/13/1972
Date Admitted	02/05/2014

Unstructured Data

Information that is difficult to organize using traditional mechanisms

"The patient came in complaining of chest pain, shortness of breath, and lingering headaches...smokes 2 packs a day... family history of heart disease...has been experiencing similar symptoms for the past 12 hours...."

- How can the unstructured data above be stored in a structured relational db?
- How can it be stored in a unstructured db?

BIG DATA

Unstructured data

Customer id	Visit id	Date
		2-Oct 2017

Customer id	Visit id	Symptom id	Symptom
		1	Chest pain
		2	Headache

Customer id	Visit id	Info
		Date: {2-Oct 2017} Symptoms: {Chest pain, Headache}

Structured db

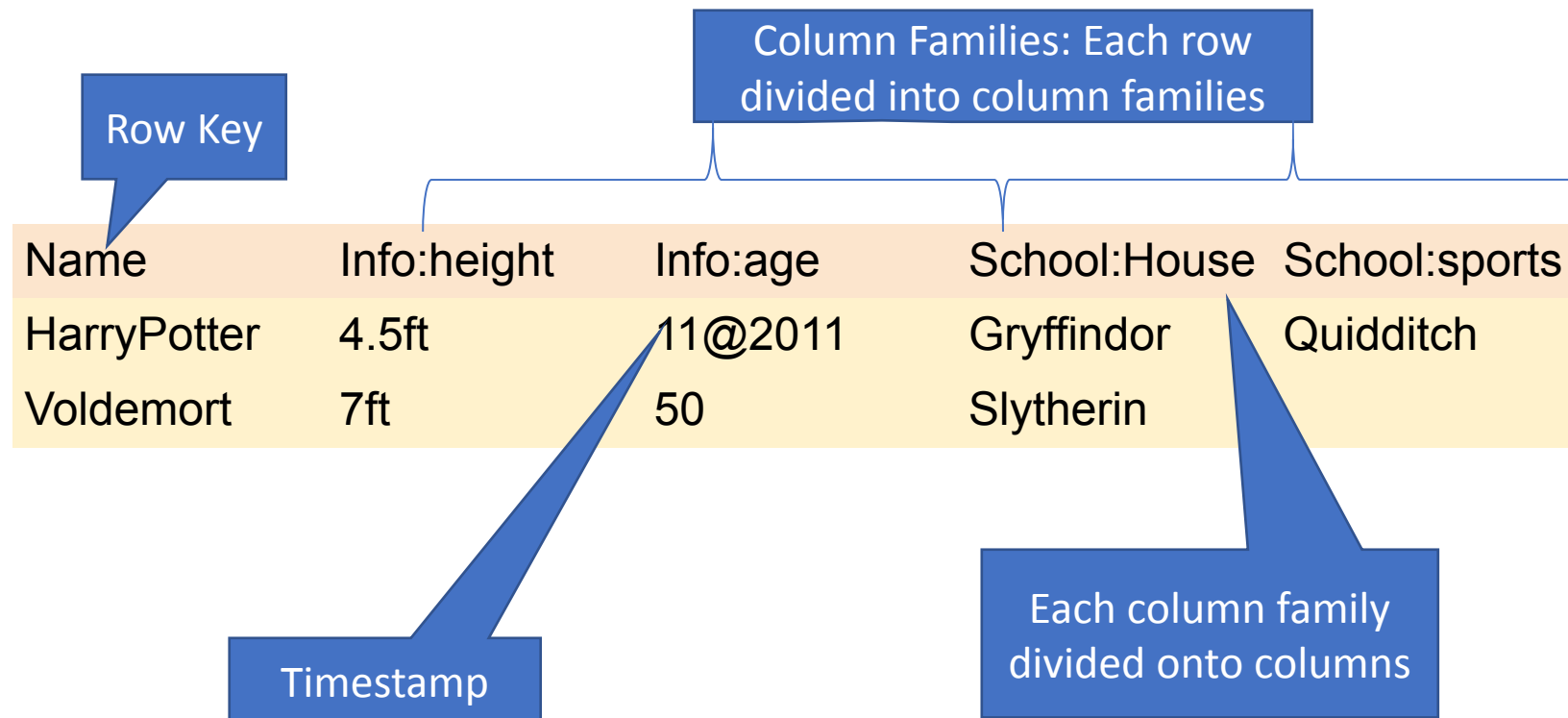
Unstructured db: simpler, more efficient

Cassandra Data Model



- **Cassandra**
 - Distributed database – **peer to peer model**.
 - built by facebook
 - Inspired by Dynamo DB
 - Same data model as Hbase – inspired by BigTable
 - Cassandra can be described as fast and easily scalable with write operations spread across the cluster.
 - The cluster does not have a master node, so any read and write can be handled by any node in the cluster.

1. Key-Value pairs



The diagram illustrates a Cassandra table structure. A table with two rows is shown. The first row contains the headers: Name, Info:height, Info:age, School:House, and School:sports. The second row contains the data: HarryPotter, 4.5ft, 11@2011, Gryffindor, and Quidditch. The third row contains the data: Voldemort, 7ft, 50, Slytherin, and Quidditch. Annotations include: 'Row Key' pointing to the 'Name' column; 'Column Families: Each row divided into column families' pointing to the entire table structure; 'Timestamp' pointing to the '@2011' part of the 'Info:age' value; and 'Each column family divided onto columns' pointing to the 'School:House' and 'School:sports' columns.

Name	Info:height	Info:age	School:House	School:sports
HarryPotter	4.5ft	11@2011	Gryffindor	Quidditch
Voldemort	7ft	50	Slytherin	Quidditch

The basic unit of storage in Cassandra is a column. A Cassandra column consists of a name-value pair where the name also behaves as the key. Each of these key-value pairs is a single column and is always stored with a timestamp value. The timestamp is used to expire data, resolve write conflicts, deal with stale data, and do other things. Once the column data is no longer used, the space can be reclaimed later during a compaction phase.

→ Column Families

- ◆ A row is a collection of columns attached or linked to a key
- ◆ A collection of similar rows makes a column family
- ◆ Each column family can be compared to a container of rows in an RDBMS table where the key identifies the row and the row consists on multiple columns.
- ◆ The difference is that various rows do not have to have the same columns, and columns can be added to any row at any time without having to add it to other rows.

“School” column family has different columns in different cells



Name	Data
HarryPotter	Info:{height:"4.5ft", age: "11@2011"} School:{House:"Gryffindor", Sports:"Quidditch"}
Voldemort	Info:{height:"7ft", age: "50"} School:{House:"Syltherin", Role:"Prefect"}

Column Families (Contd.)

Row Key	Data
HarryPotter	Info: {height:"4.5ft", age: "11@2011"} School: {House:"Gryffindor", Sports:"Quidditch"}
Voldemort	Info: {height:"7ft", age: "50"} School: {House:"Syltherin", Role:"Prefect@1980, DarkLord@1995"}

Different types of data into different column families

Single column may have different values at different timestamps

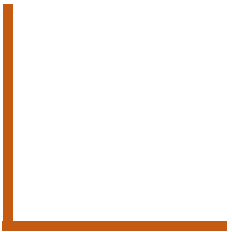
Column Families (Contd.)

Super Columns:

A super column is a special kind of column. Both kinds of columns are name/value pairs, but a regular column stores a byte array value, and the value of a super column is a map of subcolumns (which store byte array values). Note that they store only a map of columns; you cannot define a super column that stores a map of other super columns. So the super column idea goes only one level deep, but it can have an unbounded number of columns.

```
{  
  name: "book:978-0767905923",  
  value: {  
    author: "Mitch Albon",  
    title: "Tuesdays with Morrie",  
    isbn: "978-0767905923"  
  }  
}
```

Cassandra Architecture



- Cassandra DB has its origination from **Bigtable** and **Amazon's Dynamo DB**
- Its an **open-source Columnar NoSQL Database** similar to Hbase and can handle extremely large data sets and handle non-relational data.
- It's built using **Peer-to-peer Architecture** unlike the master-slave architecture of Hbase
- Cassandra supports **elastic scalability**, that is it allows increase in the number of nodes in the cluster to accommodate customers and data with linear performance scaling
- Supports prevention of data lose with failure of components by replication (similar to Hbase) of data across multiple nodes in the cluster
- Cassandra accommodates all possible data formats including: **structured, semi-structured, and unstructured**. It can dynamically accommodate changes to your data structures according to your need.
- Cassandra supports properties like **Atomicity, Consistency, Isolation, and Durability (ACID)**

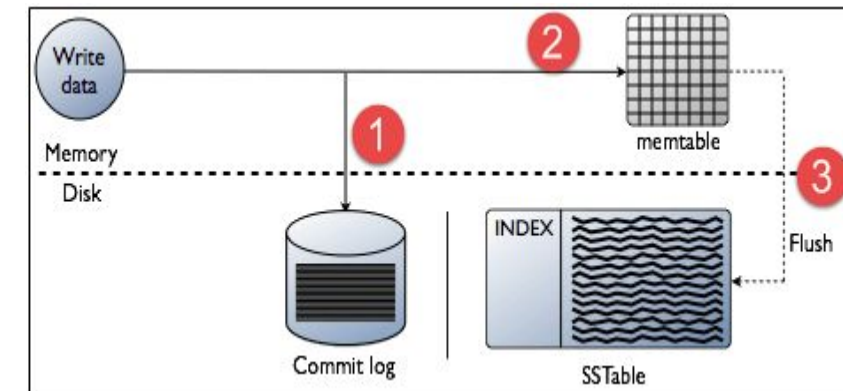
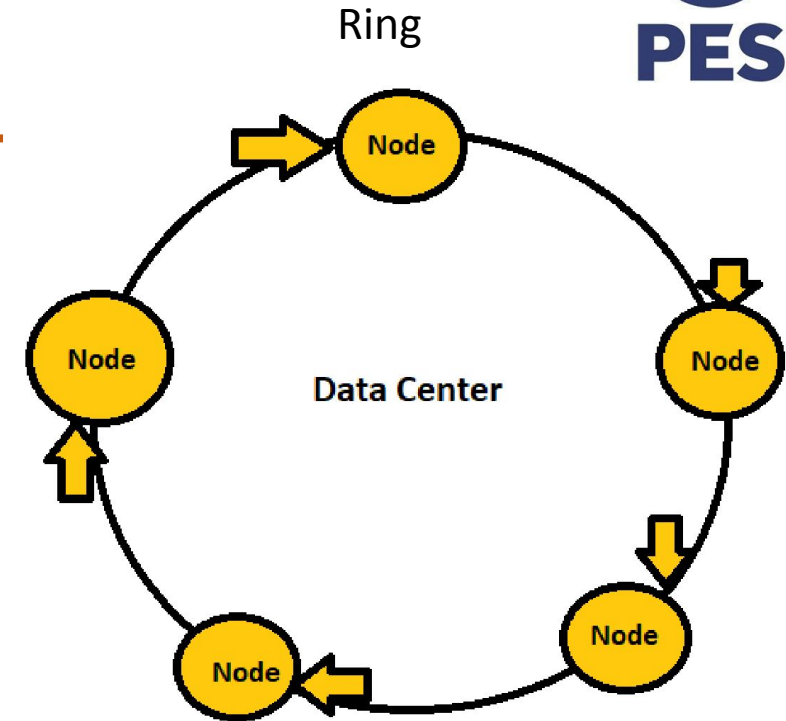
- Cassandra is probably not the best choice if only need to run a single node.
- Cassandra database is specifically designed to be distributed over several machines operating together that appear as a single instance to the end user.
- So the outermost structure in Cassandra is the cluster, sometimes called the ring, because Cassandra assigns data to nodes in the cluster by arranging them in a ring.

BIG DATA

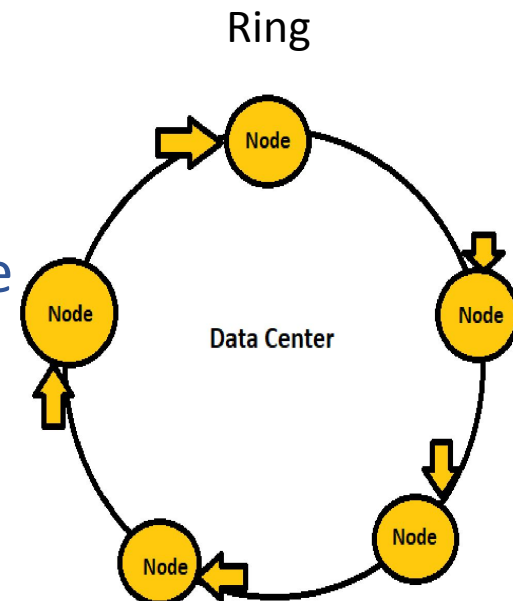
Cassandra Architecture – Peer to Peer



- Cassandra has **peer-to-peer distributed system** across its nodes, and data is distributed among all the nodes in a cluster.
 - All the nodes in a cluster play the same role. Each node is independent and at the same time interconnected to other nodes.
 - Each node in a cluster can accept read and write requests, regardless of where the data is actually located in the cluster.
 - When a node goes down, one or more of the nodes in a cluster act as replicas for a given piece of data, and the read/write requests are supported.



- Cassandra organizes data into partitions. Each partition has a partition key and consists of multiple columns. Partitions are stored on a node. Nodes are generally part of a cluster where each node is responsible for a fraction of the partitions.
- **Cassandra differs from Hbase in**
 - Request coordination happens over a partitioned dataset – no Master
 - Cassandra nodes are organized as Rings, and Ring membership and failure detection is taken care by the nodes as there is no Master
 - Local persistence (storage) engine – **does not rely on HDFS** as in HBase
 - Cassandra Shell has a specific **Query language CQL** which is richer in functions than the Hbase Query language



Where are column-family databases not a good fit?



- Event Logging: Column-family databases with their ability to store any data structures are a great choice to store event information, such as application state or errors encountered by the application. Within the enterprise, all applications can write their events to Cassandra with their own columns and the rowkey of the form appname:timestamp. Since we can scale writes, Cassandra would work ideally for an event logging system
- Content Management Systems, Blogging Platforms: Using column families, you can store blog entries with tags, categories, links, and trackbacks in different columns. Comments can be either stored in the same row or moved to a different keyspace; similarly, blog users and the actual blogs can be put into different column families.

When Not to Use Cassandra?



- There are problems for which column-family databases are not the best solutions, such as systems that require ACID transactions for writes and reads.
- If you need the database to aggregate the data using queries (such as SUM or AVG), you have to do this on the client side using data retrieved by the client from all the rows.
- Cassandra is not great for early prototypes or initial tech spikes:
- During the early stages, we are not sure how the query patterns may change, and as the query patterns change, we have to change the column family design.



THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu



BIG DATA

Hadoop Ecosystem: Flume and HIVE

Prafullata Kiran Auradkar

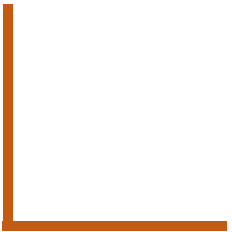
Department of Computer Science and Engineering

prafullatak@pes.edu

Acknowledgements:

Significant information in the slide deck presented through the Unit 1 of the course have been created by **Dr. K V Subramaniam's** and would like to acknowledge and thank him for the same. There have been some information which I might have leveraged from **Dr. H L Phalachandra's** slide contents too. I may have supplemented the same with contents from books and other sources from Internet and would like to sincerely thank, acknowledge and reiterate that the credit/rights for the same remain with the original authors/publishers only. These are intended for classroom presentation only.

Case Study : HIVE

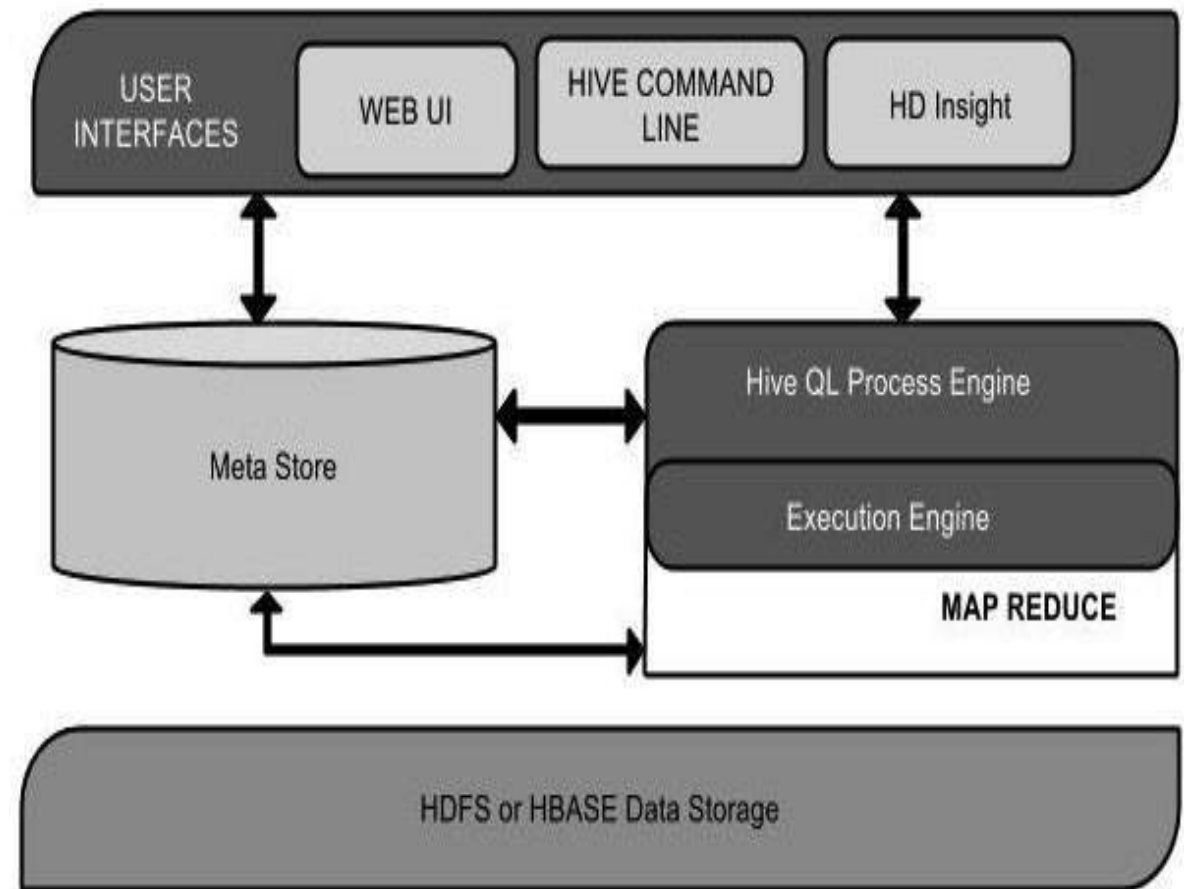


- Hive is the open source data warehouse infrastructure system to process structured data in Hadoop.
- Hive is similar to SQL and thus provides an easier and simpler way for querying, analysing, summarizing and managing structured data in HDFS, while being built on top of MapReduce and Hadoop
- Initially developed by Facebook infrastructure team for supporting
 - Structured logs with rich data types (structs, lists and maps)
 - A user base wanting to access this data in the language of their choice
 - A lot of traditional SQL workloads on this data (filters, joins and aggregations)
 - Other non SQL workloads
- The Hive process is as seen below:



Hive is a data warehousing tool that sits on top of Hadoop, allowing users to analyze large datasets using a familiar SQL-like language.

- **Foundation:** Hive operates on data that is already stored in the Hadoop Distributed File System (HDFS).
- **HiveQL:** Users write queries in HiveQL, a language that is very similar to standard SQL.
- **Metastore:** This is a central repository that stores the schema (metadata) for the data in HDFS, which is what allows Hive to interpret the data as structured tables.
- **The Engine:** Hive translates a HiveQL query into one or more MapReduce jobs, which are then executed on the Hadoop cluster to produce the result.



Advantages

- It removes the need for expertise in Java or a programming language to use Hadoop MapReduce
- It provides SQL-like queries (i.e., HQL) that are implicitly transformed to MapReduce or Spark jobs
- It is capable of analysing large datasets stored in HDFS.
- It allows different storage types such as plain text, RCFile, (Record Columnar File) and HBase.
- It can operate on compressed data stored in the Hadoop ecosystem.
- It supports user-defined functions (UDFs) where user can provide its functionality.

Limitations

- Hive is not capable of handling real-time data.
- It is Not a full database. Hive does not provide update, alter and deletion of records in the database.
- It is not designed for online transaction processing.
- It is Not developed for unstructured data
- Hive queries contain high latency

WebUI/Hive CLI - WebUI is a web based GUI and the Hive CLI (Command Line Interface) is a shell where we can execute Hive queries and commands.

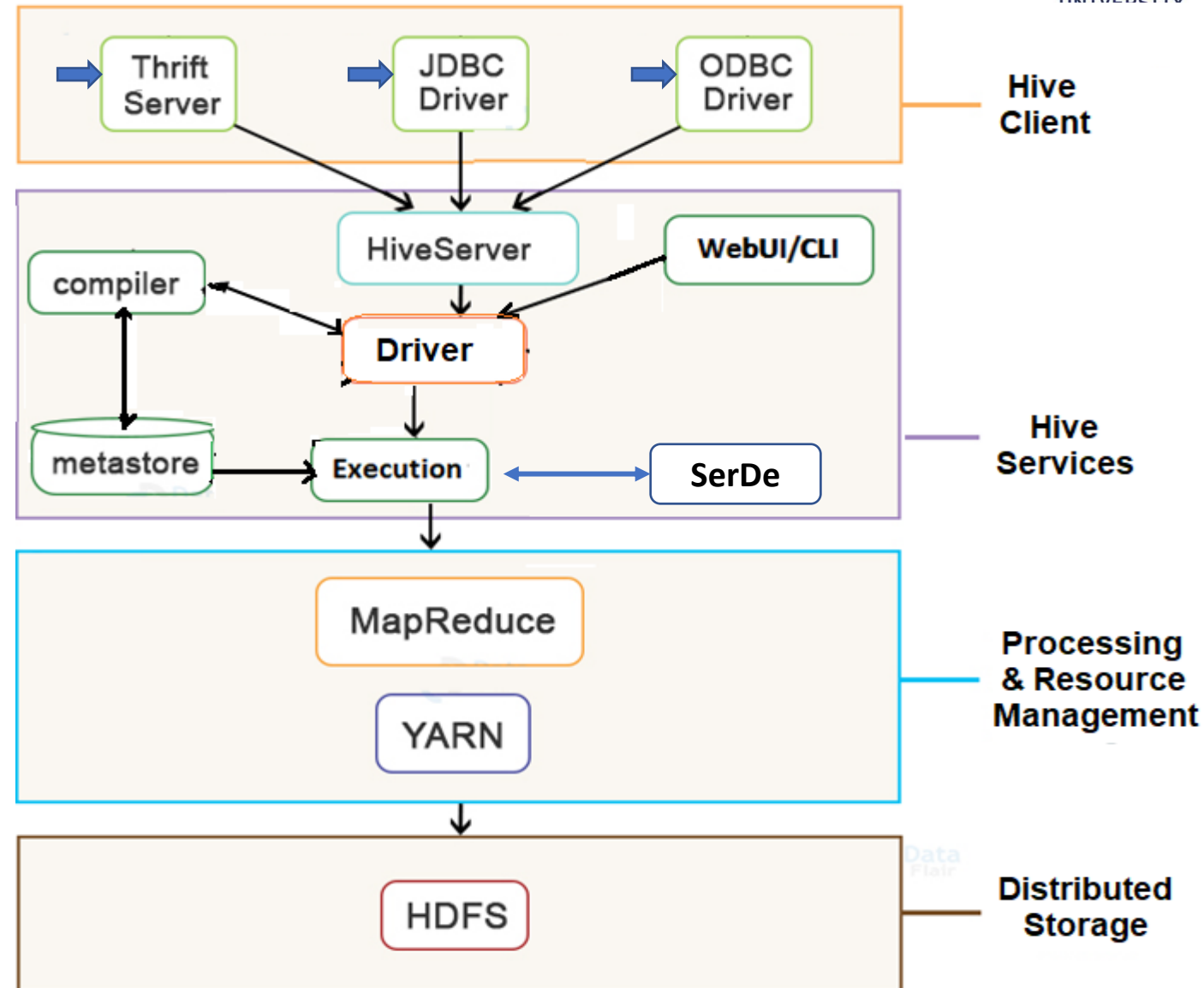
Thrift Server - It is a cross-language service provider platform that serves the request from all those programming languages that supports Thrift.

JDBC Driver - It is used to establish a connection between hive and Java applications. The JDBC Driver is present in the class `org.apache.hadoop.hive.jdbc.HiveDriver`.

ODBC Driver - It allows the applications that support the ODBC protocol to connect to Hive.

SerDe: serialization, de-serialization
Read data from disk, convert to Hive format
Reverse during writes. (Thrift, JSON)

Hive Driver - It receives queries from different sources like web UI, CLI, Thrift, and JDBC/ODBC driver. It transfers the queries to the compiler.



HIVE Components

Hive MetaStore - It is a central repository that stores all the structure information of various tables and partitions. It also includes metadata of column and its type information, the serializers and de-serializers which is used to read and write data and the corresponding HDFS files where the data is stored.

Hive Compiler - The purpose of the compiler is to parse the query and perform semantic analysis on the different query blocks and expressions. It creates the plan for execution of the query while communicating with the metastore for the metadata. It converts HiveQL statements DAG of MapReduce jobs.

Hive Execution Engine – The execution engine executes the incoming tasks in the order of their dependencies.

Hive Server - It is referred to as Apache Thrift Server. It accepts the request from different clients and provides it to Hive Driver.

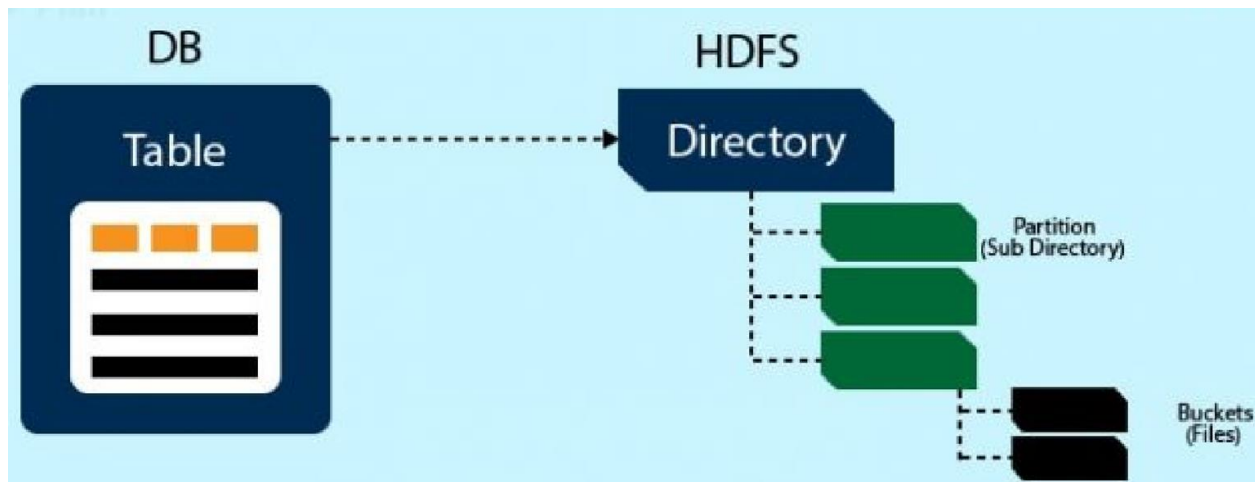
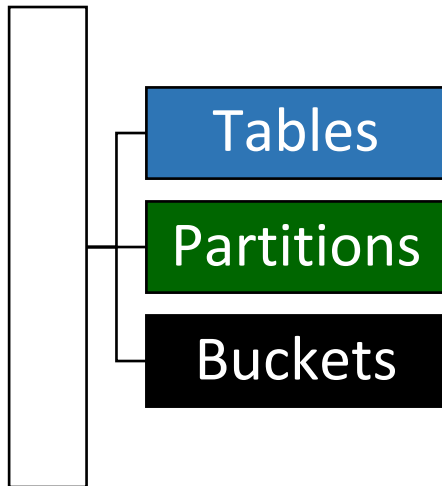
Hive Client

Hive allows writing applications in various languages, including Java, Python, and C++. It supports different types of clients and drivers to communicate with these clients. For Thrift a cross-language service provider platform, it supports with a **Thrift Server**, for Java related applications, it provides **JDBC drivers**, for other applications that support ODBC protocol, **ODBC drivers** to connect to Hive.

Hive Services:

All Client interactions with Hive is performed through **Hive Services**. Client interactions whether through **WebUI or CLI** acts as a Hive Service. All drivers communicate through **HiveServer**. **Driver** processes the requests from different applications and gets the **compiler** to convert this into an actionable plan. These are optimized through an **optimizer** and passed on to the **execution** engine using **SerDe** which helps in serialization and writing different format data into disks

Hive data is typically stored as HDFS files and can be categorized into three granular levels



Table

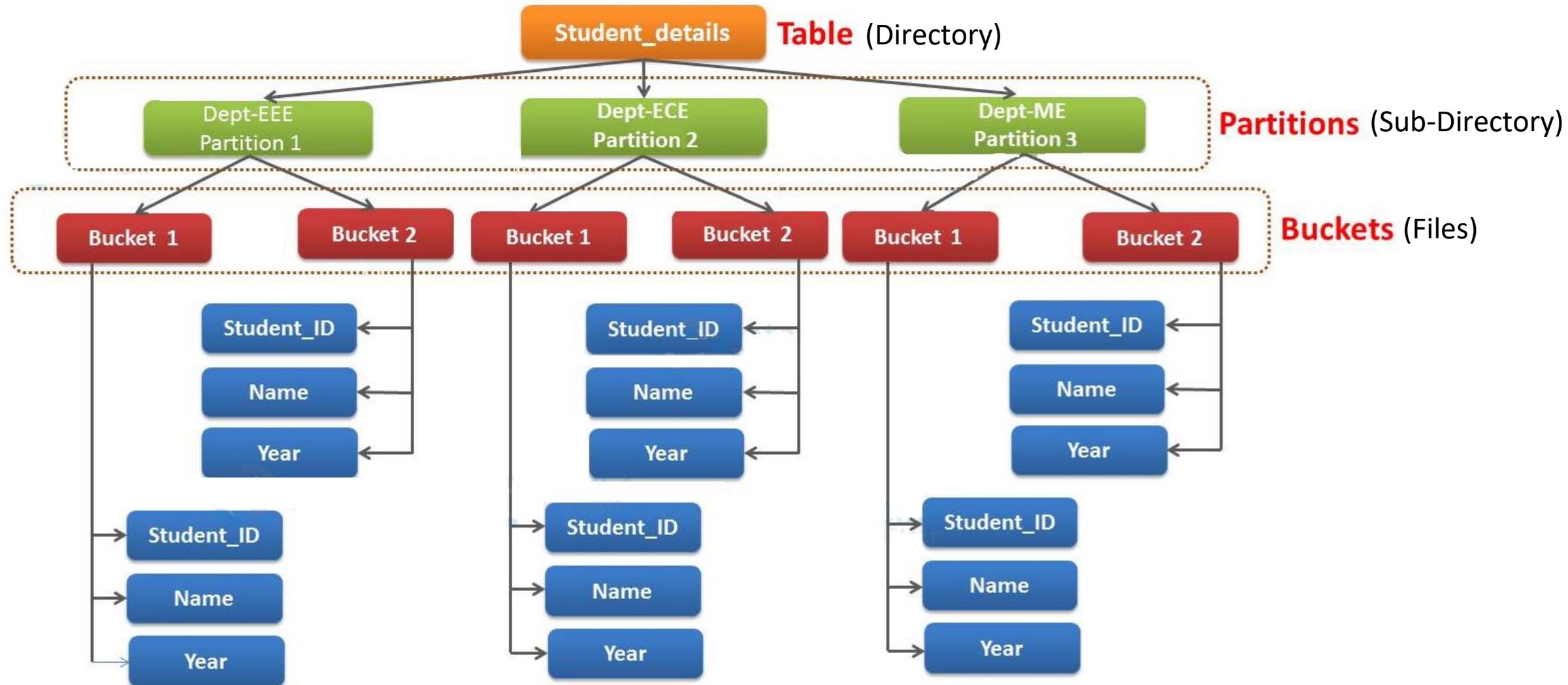
- similar to table in relational database
- Mapped to HDFS directory
- Operations like Filter, project, join and union operations can be performed on tables
- For scalability, tables divided into multiple files and directories

Partition

- Tables are organized onto partitions based on a column or partition key for grouping similar data
- Implemented as an HDFS subdirectory

Bucket

- Subdivision of partition
- Divided based on the hash function of a specified column in the column to enhance query efficiencies



It requires three software packages:

- **Java Development kit for Java compiler (javac) and interpreter**
- **Hadoop**

(Hadoop installation is already done!!!)

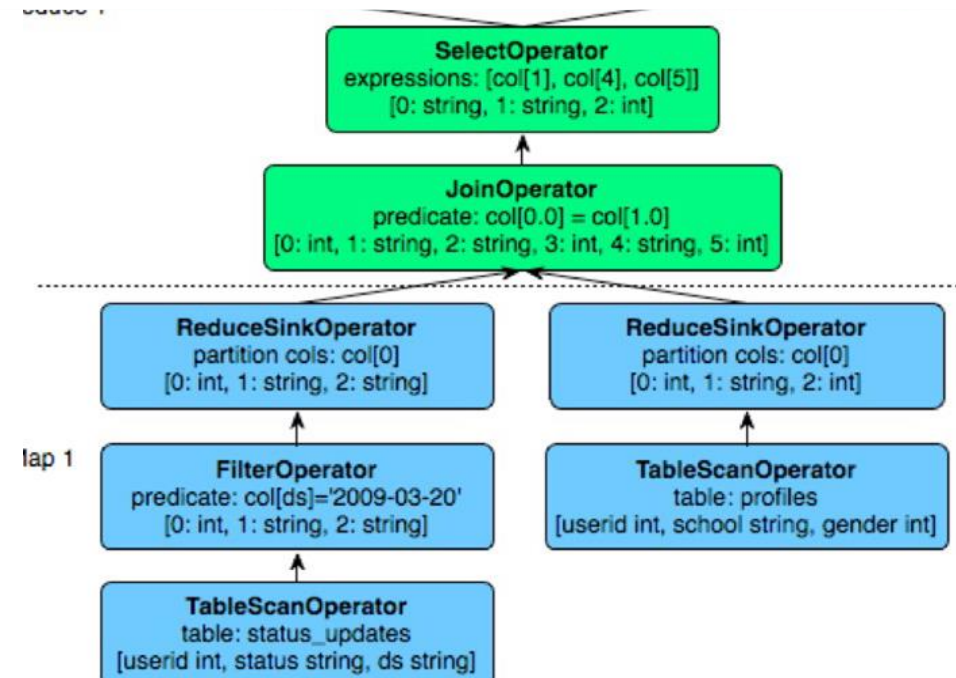
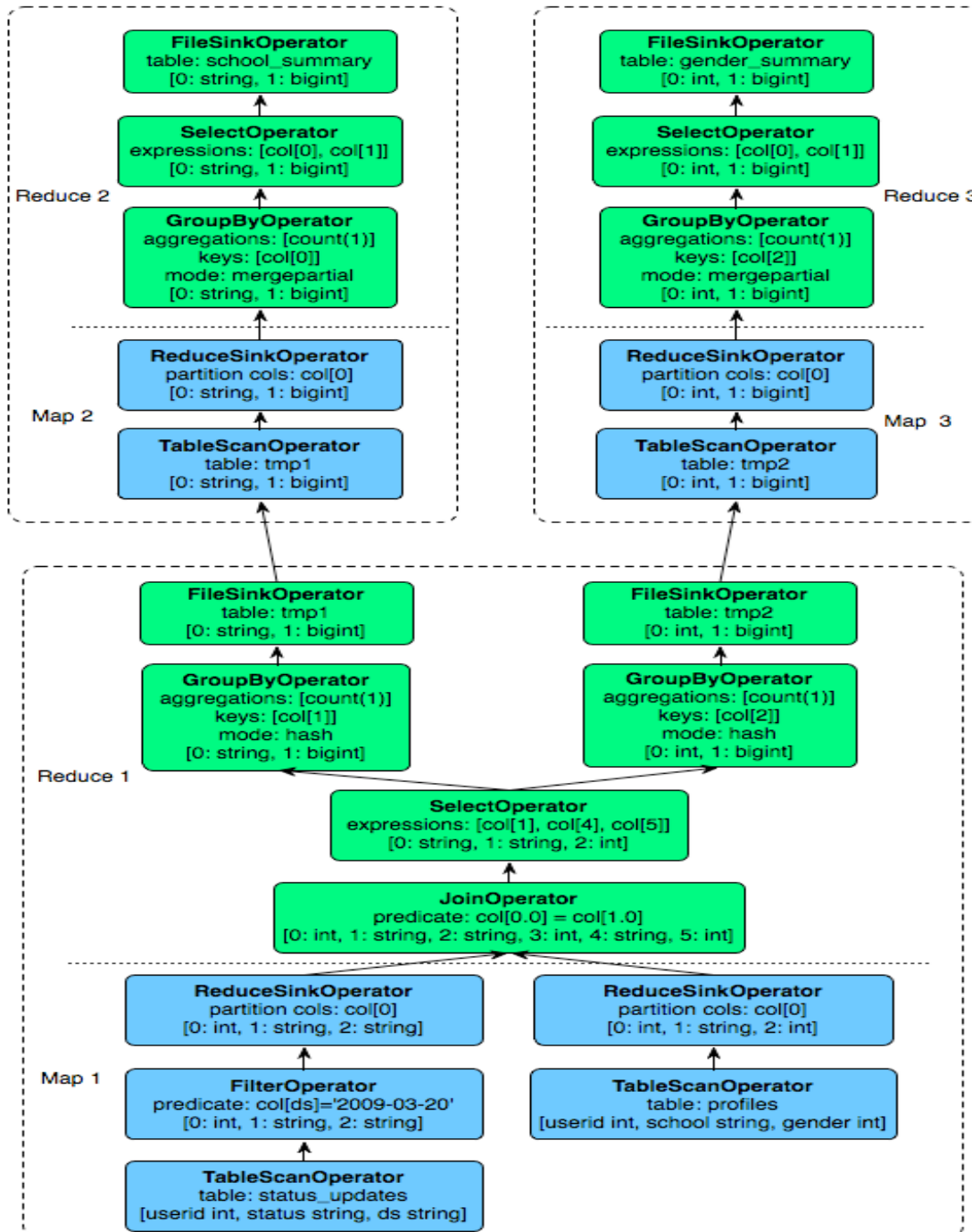
- Compatible version of Hive with Java- Hive 1.2 onward supports Java 1.7 or newer.

```
FROM (SELECT a.status, b.school, b.gender
      FROM status_updates a JOIN profiles b
      ON (a.userid = b.userid and
          a.ds='2009-03-20' )
      ) subq1
INSERT OVERWRITE TABLE gender_summary
      PARTITION(ds='2009-03-20')
SELECT subq1.gender, COUNT(1) GROUP BY subq1.gender
INSERT OVERWRITE TABLE school_summary
      PARTITION(ds='2009-03-20')
SELECT subq1.school, COUNT(1) GROUP BY subq1.school
```

HIVE compilation example BIG DATA

```
FROM (SELECT a.status, b.school, b.gender
      FROM status_updates a JOIN profiles b
      ON (a.userid = b.userid and
          a.ds='2009-03-20' )
    ) subq1
```

```
INSERT OVERWRITE TABLE gender_summary
    PARTITION(ds='2009-03-20')
SELECT subq1.gender, COUNT(1) GROUP BY subq1.gender
INSERT OVERWRITE TABLE school_summary
    PARTITION(ds='2009-03-20')
SELECT subq1.school, COUNT(1) GROUP BY subq1.school
```





THANK YOU

Prafullata Kiran Auradkar
Department of CSE
prafullatak@pes.edu